

# Site Monitor

Enterprise Monitoring Platform — User Guide

Version: 20.128.0

Document date: 2026-10-13

Language: English

# Site Monitor — Enterprise Monitoring Platform

Version `20.128.0` · August 2026 · English

## Table of Contents


1. Executive Summary
2. The Problem
3. What the Platform Does
4. Architecture and Technology
5. System Topology
6. Database Schema
7. The Certificate Check Pipeline
8. Alerting and Escalation
9. Notifications
10. Monitor Types
11. Maintenance Windows
12. Security Model
13. Roles and Permissions
14. Screens and Actions
15. Operational Procedures
16. Deployment and DevOps
17. Resilience and Availability
18. Configuration Reference
19. Release Information
20. Production Readiness Checklist
21. Glossary


## 1. Executive Summary

**Site Monitor** watches your organisation's public and internal services from a single pane of glass. At its core sits SSL/TLS certificate lifecycle management, and around that core we have built HTTP, port, DNS and ping availability checks, page integrity auditing, synthetic (k6) journey tests and domain registration tracking. It spots problems before you do, raises an alert with the team that owns the service, and keeps an audit trail of the whole thing.

When a certificate expires unexpectedly, customer traffic stops, browsers throw security warnings and you have a compliance problem on your hands. Site Monitor removes that scenario in three steps: it scans every domain in your inventory hourly, it escalates to the owning team 30, 15 and 7 days before expiry, and it keeps reminding them daily until the renewal is done.

Different people use the platform for very different reasons:

| Who | What they use it for |
|---|---|
| Operations and infrastructure teams | Certificate and availability monitoring, coordinating renewals |
| Application development teams | Watching their own services, writing the weekly report |
| IT security | Auditing, spotting weak algorithms, compliance evidence |
| Managers (product owners, heads of department, C-level) | Overall health, escalated alerts, approving reports |
| System administrators | Installation, configuration, users and permissions |

---

# 2. The Problem

This section explains why Site Monitor exists: where manual certificate tracking breaks down, and which gap automated monitoring fills. It is the context you want if you are evaluating the platform or picking up the process from someone else.

## 2.1 Certificates Multiply Quietly

A modern organisation runs dozens — often hundreds — of active SSL/TLS certificates. They come from different certificate authorities, they are deployed across servers, load balancers and CDN layers, their validity ranges from 90 days to two years, and they belong to different teams. One missed expiry date is a customer-facing outage.

## 2.2 Why Spreadsheets Fail

If you have ever tried to track certificate expiry in a spreadsheet or a calendar reminder, you already know how it ends. It depends on someone remembering. Knowledge walks out of the door when people move on. Nobody ever checks the certificate chain or the revocation status. And there is no live view — you find out when customers complain.

## 2.3 What Changes

```
Reactive                          →  Proactive
──────────────────────────────────────────────────────
Find out when it expires             Find out 30 days ahead
Someone remembers to check           Automatic hourly scan
Nobody is quite sure who owns it     Ownership sits with a team
A tangle of email threads            Structured escalation
No record of what happened           A complete audit trail
```

---

# 3. What the Platform Does

This section gives you the end-to-end picture. Certificates are the core; each of the other monitor types is covered in detail in 10. Monitor Types.

## 3.1 Automatic Certificate Scanning

Site Monitor scans every active domain in the inventory once an hour. For each one it performs a real TCP and TLS handshake — so you see the certificate your server is actually serving right now, not a theoretical value from a register. A parallel worker pool gets through large inventories in minutes.

Every scan captures and stores:

| Field | What it tells you |
|---|---|
| Subject / Issuer | Who the certificate is for and who signed it |
| Validity dates | Not Before / Not After |
| Days remaining | The number every alert threshold is measured against |
| SAN list | Every hostname the certificate covers |
| Chain status | Whether the intermediate chain is complete and valid |
| Revocation status | The verdict from OCSP, falling back to CRL |
| Trust status | Whether the chain terminates in a trusted root |
| Deployment status | Whether the served certificate matches the one you expected |
| Fingerprint and serial | SHA-256 fingerprint and serial number |
| Key and signature | Algorithm and key size |
| Key usage | Key Usage and Extended Key Usage extensions |
| Response time | How long the handshake took |

**Tracking a certificate from a file (6 October 2026).** Certificates that cannot be reached over the network are tracked too: keystore / truststore JKS files on OpenShift, `.pem` files returned by a CA, PFX files with a completed chain. You upload the file from the **Uploaded certificates** page or through the **Add certificate from file** option of the "Add domain" button. Supported formats: PEM / CRT / CER (Base64 or DER), DER, P7B / P7C, PFX / P12 (with the password), JKS / JCEKS / BKS, a ZIP holding several certificates, and pasted PEM text. Do not upload a CSR (certificate request) or a private key on its own; upload the certificate the CA returned or the PFX / JKS with the completed chain. The upload is analysed: validity dates, subject / SAN, key and signature algorithm, chain completeness, trust status and warnings (expired, not yet valid, self-signed, weak key, CSR or private key found …). Several CA certificates in one truststore can be tracked in one go. Tracking is identical to network-scanned certificates: the same thresholds and criticality tiers, the same alert / escalation / notification flow, dashboard, warnings, renewal advice, forecast, reports and inventory fields. These records carry an "Uploaded" badge. Renewing means uploading a new version: tracking moves to the new certificate, older versions are never deleted and stay visible in the **Versions** tab of the certificate window (fingerprint, validity, uploader, whether the key changed, SAN differences, PEM download). When a file holds pieces of the same chain (root, intermediate, leaf), the certificate at the end of the chain is tracked as ONE record and the others are shown as its chain; the SSL tab of the certificate window draws that chain with the same cards as network-scanned certificates, and its "Hierarchy (browser style)" switch (also "View" on the Versions tab) shows it root-first like a browser, with the selected certificate's details and a per-certificate PEM download. The tracking name can be changed later from Edit (versions and history move to the new name). If a renewal uploads the same certificate as the current version, "Upload anyway" stores it as a new version; older versions can be deleted permanently from the Versions tab (the current one cannot). The file is opened in your browser: private keys and the PFX/JKS password never leave it, only the public certificates are sent to the server and only the public chain is stored. BKS cannot be opened in the browser; convert it to PEM with `keytool -exportcert -rfc` and upload that. Obtaining the certificate from the vendor stays outside Site Monitor; only the expiry is tracked here.

## 3.2 Chain and Revocation Checking

A certificate that has not expired can still be broken. Site Monitor validates the full chain, then asks whether the certificate has been revoked — OCSP first, CRL as a fallback. CRL responses are cached, so a slow distribution point does not drag the whole sweep down.

### 3.3 Deployment Compliance

If you record the fingerprint or subject you expect on an inventory entry, every scan checks the served certificate against it. This catches the classic failure where a renewal is issued but never actually deployed to one node behind a load balancer.

### 3.4 Per-Domain Proxy Routing

Some targets sit behind a WAF that refuses traffic from the monitoring pod. Rather than routing everything through a corporate proxy — which would break the checks that work perfectly well today — you flip a single switch on the inventory entry for that one domain.

### 3.5 Criticality Tiers

Every inventory entry carries a tier from 1 to 4: customer-facing production, internal production, UAT and pre-production, and development or sandbox. The tier drives sort order, the criticality badge on the card, and which contacts get pulled into an escalation.

### 3.6 Beyond Certificates

Certificate monitoring answers "is the lock sound?". The rest of the family answers "is the door open, and is what is behind it correct?" — HTTP, port, DNS, keyword, ping, page integrity, synthetic journeys and domain registration. Each has its own tab and its own schedule.

### 3.7 Alert Management at a Glance

An alert is not a one-shot email; it has a lifecycle:

| Stage | What happens |
|---|---|
| Detection | A sweep finds a problem and confirms it over several consecutive checks |
| Alert raised | An event is recorded with a level, a type and an owning team |
| Notification | Email and, if configured, a Teams or Slack webhook |
| Acknowledgement | Someone marks it as seen, with a mandatory note; daily reminders stop |
| Reminder | Unacknowledged alerts are re-sent once a day |
| Resolution | The problem is fixed, the alert closes with a note, and a resolution notice goes out |

### 3.8 Team Ownership

Everything belongs to a team: inventory entries, monitors, weekly reports and alerts. That single decision is what makes escalation work — an alert always knows who should be looking at it.

### 3.9 Renaming a Domain

When a domain is renamed, its entire history moves with it in one atomic operation. Checks, alerts, notifications and notes all follow, so you keep the trend line instead of starting a new one.

### 3.10 Directory Integration

Site Monitor authenticates against your local account store or against Active Directory over LDAP. Directory users are provisioned on first sign-in, complete with their manager relationship, which the escalation logic can then use.

### 3.11 Weekly Reports with Approval

Teams write a short weekly operations report; the product owner approves or returns it. Approval works from the screen or straight from a link in the email, and a reminder goes out on Friday morning for anything still outstanding.

### 3.12 Permissions and the SQL Playground

Access is governed by a permission matrix — a resource key crossed with an action — layered on top of system roles and team scope. Global administrators additionally get a read-mostly SQL console for the questions the screens do not answer.

---

## 4. Architecture and Technology

Site Monitor ships as a single container image. The React front end is served as static content from inside the Spring Boot jar, so there is no separate web server to operate, and PostgreSQL is the only system of record.

### 4.1 Back End

| Layer | Technology |
|---|---|
| Runtime | Java 25 (LTS) |
| Framework | Spring Boot 4.1 (Spring Framework 7, Jakarta EE 11) |
| Persistence | Spring Data JPA over Hibernate 7 |
| Database | PostgreSQL |
| Cryptography | BouncyCastle for chain, OCSP and CRL handling |
| DNS | dnsjava, which exposes TTL, response time and authoritative servers |
| HTML parsing | jsoup, for the page integrity resource inventory |
| Metrics | Micrometer, exposed for Prometheus |
| Sessions | Spring Session, persisted to the database |
| Passwords | BCrypt |

Authentication is deliberately a custom interceptor rather than a full security filter chain. It keeps the request path short and the behaviour explicit, and it is why the single-active-session rule and the progressive lockout logic are straightforward to reason about.

### 4.2 Front End

| Layer | Technology |
|---|---|
| Framework | React 18 |
| Build | Vite 5 |
| Charts | Recharts |
| Icons | Lucide |
| Dates | date-fns |
| Client-side export | jsPDF for on-screen PDF exports |

The interface is a single-page application with no client-side router: the active tab lives in the query string, which is what makes every screen shareable as a link. Both languages and both themes are built in.

### 4.3 Concurrency and Performance

Checks run on a dedicated thread pool, sized by configuration, and are deliberately kept off the HTTP request threads. The HTTP thread ceiling is set lower than the framework default so that it stays in proportion to the database connection pool — an oversized thread pool simply queues on the database and wastes memory on stacks.

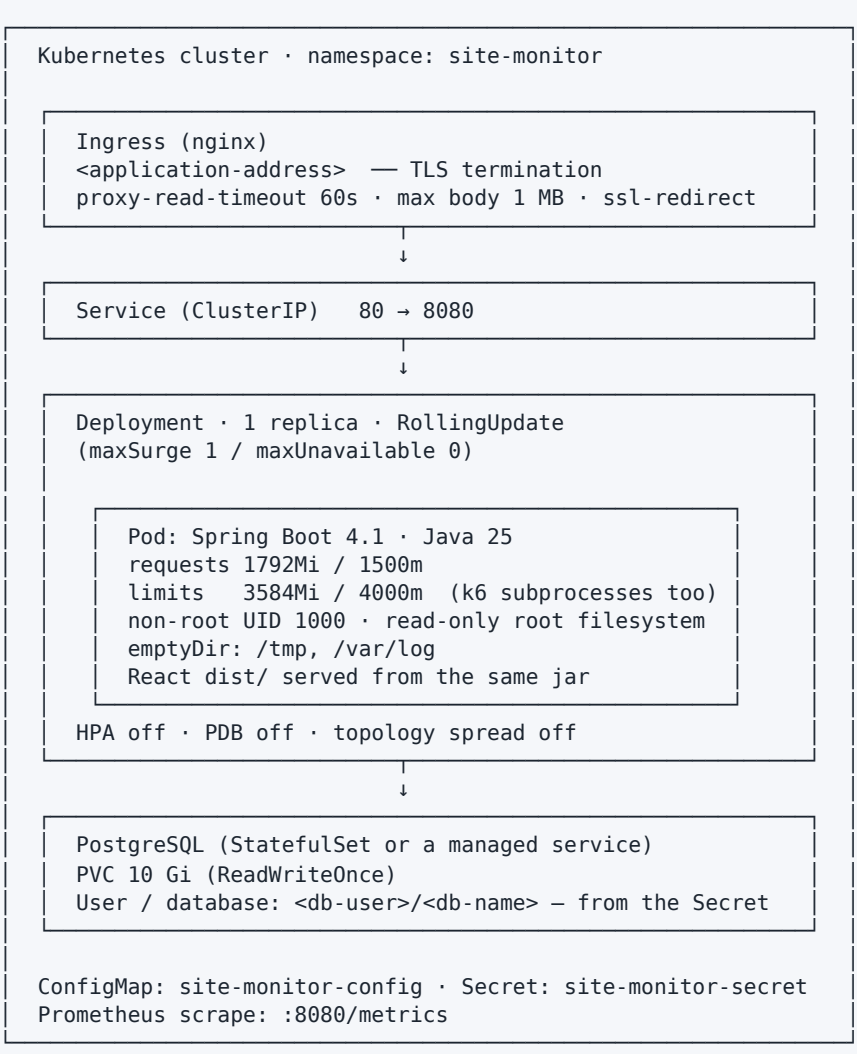
Long-running work never blocks a user. Manual checks are dispatched asynchronously, email delivery retries in the background, and expensive lookups such as CRL fetches are cached.

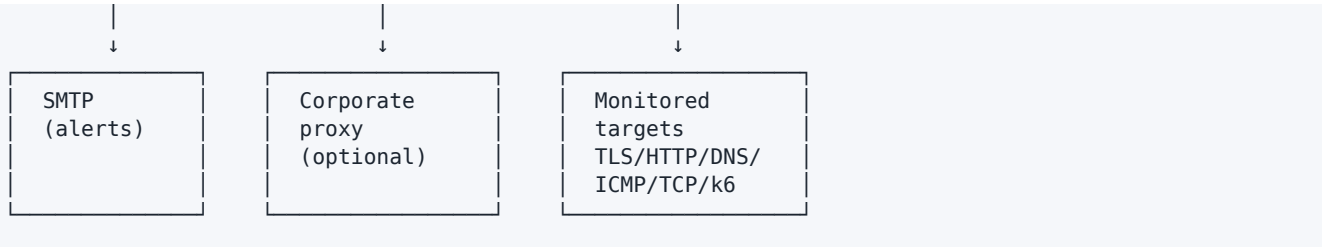
---

## 5. System Topology

This section shows how the platform sits in production and in local development. The production deployment is built around a **single pod**: roughly 100 concurrent users and 200 to 1,000 monitored targets are handled by one JVM, and you scale it vertically. Even so, nothing stateful lives in the pod's memory — sessions and scheduler locks go to the database — so a restart loses nothing, and moving to horizontal scaling later is a configuration decision rather than a rewrite.

### 5.1 Production on Kubernetes



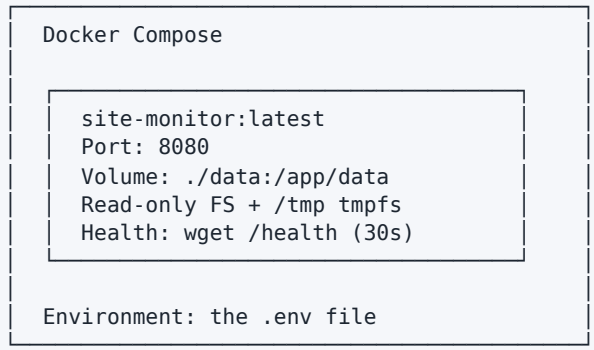



The resource limits are not arbitrary. The memory ceiling is 3.5 GiB because synthetic monitoring launches k6 subprocesses outside the JVM on every run, and their resident memory sits on top of the heap. The JVM options size the heap as a percentage of the container limit and tell it to exit immediately on an out-of-memory error, so a pod that runs out of memory restarts quickly rather than hanging in a half-working state.

The database user and database name are environment-specific. The `<db-user>/<db-name>` values come from the `.env` file — from a Secret on Kubernetes — and are never baked into the code.

The chart also supports horizontal scaling: replica count, autoscaling, pod disruption budget and topology spread are all present and switched off. If you turn them on, the multi-pod behaviour described in 17. Resilience and Availability takes over; on a single pod those same mechanisms run harmlessly.

## 5.2 Local Development with Docker Compose




## 5.3 The Distributed Scheduler Lock

If more than one pod is running, only one of them should be sweeping at any moment. Site Monitor uses a database-backed lock for this:

```
scheduler_lock table:
  name         → "cert-check"
  locked_by    → "hostname-uuid8"
  locked_until → expiry timestamp (TTL: 10 minutes)

Flow:
  Pod starts    → stale locks from the same host are cleared
  Sweep is due  → INSERT INTO scheduler_lock (unique constraint)
    ├─ Success        → this pod runs the sweep
    └─ Duplicate key  → another pod has it, skip
  Sweep ends    → DELETE FROM scheduler_lock
```

If a lock ever gets stuck, you can force-release it from the **System Health** screen.

---

# 6. Database Schema

PostgreSQL is the single system of record. There are more than seventy tables; this section groups them by purpose so that "where is this stored?" has one answer. The column names are taken from the live schema, so you can use them directly in the SQL Playground.

## 6.1 How the Schema Evolves

There is **no Flyway or Liquibase** in Site Monitor. The schema moves forward through two mechanisms:

1. Hibernate's `ddl-auto=update` creates new entities and new columns.
2. A start-up routine applies idempotent `ALTER TABLE` and `CREATE TABLE` statements, swallowing "already exists" errors. Column widenings, indexes and the tables that have no entity arrive this way.

You need to understand the operational consequences of that choice. **There is no rollback** — roll a release back and the schema stays where it is (a surplus column is harmless, a missing one is not). `ddl-auto` will **never drop a column, rename one, or backfill a default**; those three are patched by hand. And on a large schema the first start-up after an upgrade spends real time validating and patching, which is why the pod's start-up probe budget is around 300 seconds.

Four tables are not managed as entities: the scheduler lock (raw JDBC), the daily and hourly rollup tables (native SQL), and the two session tables that Spring Session creates for itself.

## 6.2 Users, Teams and Permissions

| Table | Purpose | Key columns |
|---|---|---|
| `app_users` | Accounts and the profile pulled from the directory | `username`, `password_hash`, `system_role`, `org_role` (+ `*_locked`), `team_id`, `auth_source`, `manager_id`, `lockout_until`, `permanent_lock`, `must_change_password`, `temp_password_expires_at`, `active_session_id` |
| `app_user_teams` | Multi-team membership | `user_id`, `team_id` |
| `teams` | Team definitions | `name`, `description`, `active`, `leader_id`, `email`, `weekly_reminder_enabled`, `weekly_availability_enabled` |
| `permission_grants` | Role × resource × action matrix | `role`, `resource_key`, `action`, `allowed`, `updated_by` |
| `password_history` | Prevents password reuse | `user_id`, `password_hash`, `created_at` |
| `remember_me_tokens` | Persistent sign-in tokens (7 days) | `token`, `user_id`, `expires_at` |
| `spring_session` / `spring_session_attributes` | Session store | Managed by the framework |

## 6.3 Monitored Targets

Each monitor type has its own target table. The shared columns are `name`, `team_id`, `group_name`, `interval_seconds`, `timeout_ms`, `confirm_attempts`, `confirm_interval_seconds`, `recovery_checks`, `recovery_interval_seconds`, `active`, `notify_email` and `tags`.

| Table | Purpose | Type-specific columns |
|---|---|---|
| `certificate_inventory` | The target list for certificate monitoring — there is no separate monitor table | `domain`, `port`, `tier`, `team_id`, `ug_team_id`, `use_proxy`, `tls_mode`, `expected_fingerprint`, `expected_subject`, thirteen operational flags, `deleted_at` |
| `http_monitors` | HTTP and website checks | `url`, `method`, `expected_status`, `follow_redirects`, `verify_ssl`, `check_ssl_errors`, `ssl_reminder_days`, `domain_reminder_days` |
| `port_monitors` | Port checks | `host`, `port`, `protocol`, `expect`, `send_data`, `ip_version`, `slow_threshold_ms`, `standalone` |
| `dns_monitors` | DNS record checks | `domain`, `record_type`, `expected_value`, `propagation_check`, `dns_change_alert_enabled`, `standalone` |
| `keyword_monitors` | Page content checks | `url`, `keyword`, `match_operator`, `match_count`, `alert_condition`, `case_sensitive`, `custom_headers` |
| `ping_monitors` | ICMP checks | `host`, `ip_version`, `packet_count` |
| `page_monitors` | Page integrity checks | `url`, `mode`, `crawl_depth`, `crawl_max_pages`, `exclude_patterns`, `slow_resource_ms`, `alert_third_party`, `alert_mixed_content`, `resource_concurrency` |
| `domain_monitors` | Domain registration checks | `domain`, `warning_days`, `critical_days`, `thresholds_csv` |
| `scripted_monitors` | Synthetic k6 journeys | `script`, `env` (encrypted), `timeout_seconds`, `slow_threshold_ms`, `use_proxy` |
| `scripted_script_versions` | Version history for k6 scripts | `monitor_id`, `script`, `created_at`, `created_by` |
| `scripted_drafts` | Editor autosaves | `monitor_id`, `user_id`, `script`, `updated_at` |
| `monitoring_groups` | Per-team monitor groupings | `type`, `name`, `name_lower`, `team_id` |
| `maintenance_windows` | Suppression windows | `all_monitors`, `targets_json`, `timezone`, `start_at`, `duration_minutes`, `recurrence`, `days_of_week`, `day_of_month` |

## 6.4 Check Results

These append-only tables grow fastest and are the main target of the retention policy (see §15.8).

| Table | Purpose |
|---|---|
| `certificate_checks` | Full certificate check history |
| `latest_checks` | One row per domain holding the most recent certificate result — this is what the screens read |
| `uptime_checks` | Availability measurements for inventory domains |
| `http_checks` · `port_checks` · `ping_checks` · `keyword_results` · `dns_records` · `domain_checks` · `page_checks` · `scripted_checks` | Raw results, one table per type |
| `page_resource_issues` | Resources that failed during a page check — only the problem ones are stored |
| `monitor_check_daily` / `monitor_check_hourly` | Daily and hourly rollups; the long-term trend survives here after raw rows are purged |

| Table | Purpose |
|---|---|
| `http_metric_minute` | The application's own per-minute latency and request counters |
| `system_heartbeat` | One health signal per minute |
| `activity_log` | The unified activity stream — one row per check, isolated by team |

## 6.5 Alerts, Incidents and Notifications

| Table | Purpose | Key columns |
|---|---|---|
| `alert_events` | Alert lifecycle | `domain`, `alert_level`, `alert_type`, `team_id`, `context_json`, `acknowledged` + `_by/_at/_note`, `resolved` + `_at/_by/_note`, `last_re_alert_at`, `realert_count`, `storm_id`, `cert_tier` |
| `alert_comments` | Comments on an alert | `alert_event_id`, `author`, `body`, `created_at` |
| `alert_thresholds` | Day thresholds | `warning_days`, `high_days`, `critical_days`, `re_alert_interval_hours` |
| `alert_storms` | Storm grouping | `scope_key`, `scope_type`, `member_count`, `root_cause`, `notified_teams`, `resolved` |
| `escalation_contacts` | Who gets notified | `user_id`, `email`, `role`, `min_alert_level`, `webhook_url`, `webhook_type`, `team_id` |
| `notification_logs` | Every delivery attempt | `alert_event_id`, `recipient_email`, `subject`, `email_status`, `webhook_status`, `trigger`, `sent_at` |
| `network_outage_events` | Bulk network failure events | `detected_at`, `error_rate`, `failed_count` |
| `incident_records` | The hand-written incident ledger | `title`, `occurred_at`, `detected_at`, `resolved_at`, `severity`, `status`, `category`, `rca_summary`, `business_impact`, `affected_app`, `sla_breached`, `duration_minutes` |
| `incident_images` / `incident_options` | Screenshots and dropdown options | |
| `login_anomaly_incident` | Failed sign-in anomalies | `opened_at`, `rule`, `resolved` |

## 6.6 Settings, Reports and Operations

| Table | Purpose |
|---|---|
| `app_settings` | Live settings — key and value, changeable without a restart |
| `smtp_settings` / `ldap_settings` | Mail and directory configuration; passwords encrypted at rest |
| `pinned_cas` | Automatically pinned certificate authorities, per host and port |
| `scheduler_lock` | The distributed scheduler lock |
| `retention_run` / `retention_run_item` | The nightly clean-up's own history — how many rows it removed from which table |
| `audit_log` | The security audit trail — hash-chained, with IP and geolocation |

| Table | Purpose |
|---|---|
| `monitor_change_log` | Monitor CONFIGURATION history — who changed which field, when, from which IP, with a full snapshot of every event (kept for 730 days) |
| `weekly_reports` / `weekly_report_images` / `weekly_report_mails` | Weekly report content, images and the archive of what was sent |
| `weekly_availability_log` | Idempotency record for the weekly availability email |
| `cert_inventory_report_log` | Delivery record for the monthly inventory report |
| `certificate_notes` / `certificate_note_revisions` | Per-domain notes and their edit history |
| `monitor_notes` / `monitor_guide` | Per-monitor notes and runbook content |
| `guide_links` | Links shown on the Renewal Guide screen |
| `diagnostic_runs` | History of diagnostic tool runs |
| `sql_query_history` | SQL Playground query history |
| `login_issue_reports` and related tables | Problem reports, screenshots and mail history |

### 6.7 Deletion Is Permanent

Since 7 October 2026, deleting a certificate (network-scanned or uploaded from a file) or a monitor is always **permanent**; there is no bin and no restore. Deleting a certificate first closes its open alerts, then removes in one transaction its check history, latest result, notes, uploaded versions, the Port/DNS monitors derived from the inventory and their checks, uptime records, weak-algorithm exception and diagnostic history. Closed alert records, the audit log and the change history stay as a trace. A deleted record's name is free at once, so a new record can use it. To stop monitoring without deleting, switch "Active" off.

```
active     BOOLEAN       -- FALSE = paused (record kept, not monitored)
deleted_at VARCHAR(255)  -- legacy bin column; no longer written
```

When upgrading to this version, records still in the old bin are permanently deleted once at the first start and their names are written to the audit log.

---

## 7. The Certificate Check Pipeline

This section walks through what happens to a certificate during the hourly sweep. Knowing the pipeline is the quickest route to understanding why an alert fired and when.

### 7.1 The Full Flow

```
Scheduled trigger (hourly, configurable cron)
        │
        ▼
Acquire the distributed lock ──── already held? ──→ skip this run
        │
        ▼
Load active inventory domains
        │
        ▼
Fan out across the check thread pool
        │
        ├──→ Per domain:
```

```
        │      1. Resolve DNS; validate every resolved address
        │      2. Open a TCP connection (direct, or via the proxy if flagged)
        │      3. Complete the TLS handshake, capture the served chain
        │      4. Read the leaf: subject, issuer, validity, SAN, key, signature
        │      5. Validate the chain and check revocation (OCSP, then CRL)
        │      6. Compare against the expected fingerprint and subject
        │      7. Work out the days remaining and the status
        │      8. Write certificate_checks + update latest_checks
        │      9. Write an activity_log row
        │
        ▼
Was this a bulk network failure? ──── yes ──→ suppress individual alerts,
        │                                     raise one network outage event
        ▼ no
Evaluate alert thresholds per domain
        │
        ▼
Raise, re-alert or resolve alerts; notify the owning team
        │
        ▼
Release the lock
```

### 7.2 Bulk Network Outage Detection

If the proportion of network-class failures in a single run crosses a configured threshold — and at least a minimum number of checks failed — Site Monitor concludes that the problem is its own egress rather than five hundred separate servers. Individual certificate alerts for that run are suppressed and one network outage event is raised instead, with an email to the system administrator. This behaviour is deliberate; please do not "fix" it by alerting on everything.

### 7.3 Retry Policy

Only network-class failures are retried, and only once. SSL, DNS and certificate errors are never retried, because they are real findings rather than transient noise.

### 7.4 Catching Up After Downtime

If the application was down when a daily notification was due, the notification is not lost. On start-up, any of that day's outstanding daily alerts are sent immediately — without waiting for a network call — and the full sweep follows. However long the outage lasted, the day's notifications still go out.

---

## 8. Alerting and Escalation

An alert is a record with a lifecycle, not a single email. This section covers how one is born, who hears about it, and how it closes.

## 8.1 Alert Lifecycle

```
Problem detected
     │
     ▼
Confirmation: N consecutive failed checks (default 3)
     │
     ▼
Alert raised  ── level: WARNING / HIGH / CRITICAL
     │           type:  what kind of problem
     │           team:  who owns it
     ▼
Notification ── email + optional webhook
     │
     ├──→ Acknowledged (with a mandatory note)
     │      daily reminders stop, the alert stays open
     │
     ├──→ Not acknowledged
     │      one reminder a day until someone acts
     │
     ▼
Recovery: N consecutive successful checks (default 3)
     │
     ▼
Alert resolved ── resolution notice sent
```

## 8.2 Alert Types

Every monitor type raises its own family of alerts: certificate expiry, broken chain, revoked or mismatched certificates; availability failures for HTTP, ping and ports; DNS resolution failures and record changes; keyword conditions; page integrity problems; synthetic journey failures; and domain registration expiry. Twenty-eight distinct types are recognised and labelled throughout the interface.

## 8.3 The Escalation Matrix

Alert levels are driven by how many days remain:

| Level | Default threshold | Who is told |
|---|---|---|
| WARNING | 30 days | The owning team |
| HIGH | 15 days | The team plus escalation contacts at that level |
| CRITICAL | 7 days | The team, escalation contacts and management |

Thresholds are configurable from the **Admin Panel**.

Escalation contacts are set up per team, and each contact only receives alerts for its own team. If none of a team's contacts matches the level, no escalation contact is added and the alert goes only to the team's own recipients (team address, notification group, push) — another team's manager or contacts are never added. Records with no team (unowned) send no notifications on any channel.

## 8.4 Recipient Routing

Who receives an alert depends on where it comes from as well as its level, and every path — the opening alert, daily reminders, escalation, resolution, manual resends and storm digests — follows the same rules:

- Standalone monitors (keyword, ping, HTTP, page, synthetic, page speed, domain, and Port/DNS monitors added by hand) belong to the team stamped on the alert when it opened. Every notification goes to that team only; the team is never taken from the certificate inventory.

- Certificate alerts, availability alerts and Port/DNS monitors derived from the inventory go to the inventory's SY and UG teams, and each of the two brings only its own escalation contacts (one email per address).
- The level rule is the same everywhere: at WARNING a standalone monitor notifies only its team; at HIGH and CRITICAL escalation contacts are added according to their own minimum level. Another team's contacts are never added, and an unowned record sends nothing.

## 8.5 Notification Triggers

Each notification records why it was sent: the initial alert, a daily reminder, a manual resend, or a resolution notice. You can see the trigger against every entry in the notification history.

## 8.6 What Acknowledgement Does

Acknowledging an alert stops the daily reminders. It does **not** close the alert — the problem is still there, someone has simply taken ownership of it. Closing happens either automatically, when the monitor recovers, or manually when you mark it resolved.

## 8.7 Alert Storm Grouping

When a lot of monitors fail at once — a shared network segment goes down, say — you do not want an email per monitor. Site Monitor tracks confirmed outages: if there is an active storm for the scope, new alerts join it, and once the threshold is crossed individual notifications are promoted into a single bulk notification. Individual alert records are still written per monitor, so history and uptime percentages are unaffected; only the notification is grouped. When the storm clears, one bulk recovery notice goes out.

**Storms are assessed per team (29 September 2026).** The threshold, the window count and the root cause are worked out only from the monitors of the team that **owns** the alert. The threshold is compared with the number of **different targets** (hosts): the availability, port and DNS alerts of one host count as a single target, and with a percentage threshold at least 3 targets are needed, so a single host failing in a small team is never treated as a storm. The summary notification (email, personal push and webhook) goes to that team alone. Another team's failures are never added to your storm, nor yours to theirs, and there is no organisation-wide "All monitors" storm notification. Each team sees only its own count, name and monitors. A storm push is resolved at the highest alert level among its members, so it never reaches a wider audience than the individual alerts would have (for instance, tiers that only receive critical alerts). An alert with no owning team never joins a storm. The 24/7 Monitoring Team receives a separate summary only for members whose monitor has 24/7 notifications explicitly switched on.

**Optional: keeping push out of the storm (3 October 2026).** By default personal push is handed over to the storm as well: while a storm lasts, member alerts aren't pushed one by one and the team gets a single summary storm push. If Settings → Alert Storm → "Don't hand push notifications over to the storm" is switched on, the storm groups email only (together with webhook and the 24/7 summary); every member alert's push — opening, level rise, daily reminder and resolution — goes out individually, in the order the alerts opened. The hourly push cap per user, team and personal quiet hours and system maintenance still apply in that mode.

**A manual check never raises an alert or a storm.** "Check now" on the monitor pages (a single monitor, or the bulk "Check Now (N)") only records the result: it won't open a new alert, escalate, re-alert or start a storm, and it doesn't advance an open alert's notification counters. If the result is healthy, an open alert can close under the normal recovery rule (the resolution notice goes only to people who were notified when it opened). If a manual check fails, the alert is raised by the next scheduled check under the usual confirmation rules. A manual check also won't send a domain threshold reminder, won't trip the synthetic monitor anomaly guard (it never switches a monitor off) and doesn't use up change detection for DNS or domain registration data (nameservers, registrar, EPP, DNSSEC): the scheduled check still sees the new value as a change against its own baseline and raises the alert.

A domain's Registration tab shows the stored data when it opens; a live lookup only runs when you press "Refresh". A bulk check only covers monitors you're allowed to run; a team-scoped administrator can't run another team's monitors.

The feature is on by default and managed from **Settings → Storm**:

| Setting | What it controls |
|---|---|
| Threshold unit | A count, or a percentage of the team's active monitors |
| Threshold value | How many simultaneous outages declare a storm |
| Window (minutes) | How close together outages have to be to count as simultaneous |
| Per group | Whether storms are evaluated within each of a team's monitor groups rather than across all of the team's monitors |
| Retention (days) | How long closed storm records are kept |

Only "down"-class alert types are counted, so time-based alerts such as expiry warnings never create a storm.

There is a second, independent suppression layer: **bulk network outage detection**. If the error rate in a sweep crosses its threshold and enough checks failed, Site Monitor decides the fault is its own egress rather than the targets. Individual certificate alerts for that run are suppressed and a single network outage event is raised instead. This is the right answer — when the pod loses its internet connection, one "egress is down" alert beats five hundred "certificate unreadable" alerts.

### 8.8 The Second Daily Check for Critical Domains

The level of a domain expiry alert mirrors the card status exactly: below the critical threshold it is CRITICAL, between critical and warning it is WARNING. On top of the morning sweep, a second sweep every afternoon re-checks only those domains that have already entered the critical band. A domain renewed that same day therefore has its alert closed without waiting until the next morning, while daily de-duplication still stops a second notification going out for one that is genuinely still critical.

---

## 9. Notifications

Site Monitor delivers over email and, optionally, a chat webhook. Every attempt on both channels is logged, so "did anyone actually get told?" is always answerable.

### 9.1 Email Design

Alert emails are built to survive corporate mail clients, which is a harder constraint than it sounds — Outlook on the desktop renders with the Word engine and ignores a good deal of modern CSS. The templates therefore use table-based layouts, solid colours and inline images referenced by content ID, so they render the same whether or not remote content is blocked.

Every alert email carries the domain, the level, the type, the days remaining, the detail table and a direct link to the relevant screen. Resolution emails come with the "back to green" branding, the total outage duration in a compact form, and a clear start-to-recovery window.

Alert and resolution emails both include a short **"why did I get this?"** block naming the team the notification is registered against. It is there for trust: a recipient who can see why a message reached them is far less likely to treat it as phishing.

## 9.2 Webhooks

Email is not the only channel. Alongside an email address, each escalation contact can carry a webhook:

| Platform | Format | Note |
|---|---|---|
| Microsoft Teams | MessageCard | The card colour follows the alert level |
| Slack | Slack message payload | Select `SLACK` as the type on the contact record |

Webhook deliveries land in the same audit trail as email: every attempt appears in the notification log with its recipient, subject and outcome. The timeout is ten seconds, and a failed webhook never blocks the email — the two channels are independent.

## 9.3 The Weekly Availability Email

Every active team that owns certificates receives a summary of its domains' availability for the previous full week (Monday to Sunday) on Monday morning. It goes out whether or not there were outages, with any affected domains highlighted. It is addressed to the team mailbox, with the team's PO, escalation (TECH) and manager (MANAGER) contacts copied in; the C-level contact is left off the routine summary.

A **detailed outage report is attached as a PDF**: every outage of the week, with charts and status colouring, and the email body announces the attachment explicitly. Managers read the summary in the body and find the detail in the attachment.

Delivery is idempotent per team and per week — a second email for the same week never goes out. Configure it under **Settings → Weekly Availability**, where you can also review past sends, preview the next one, download the outage PDF on its own, and send yourself a test.

## 9.4 Email Configuration

Mail can be configured from environment variables at start-up, but after installation it is far more practical to manage it from **Settings → SMTP**: changes there are written to the database, take effect without a restart, and the screen can send a test message to prove the settings work.

## 9.5 24/7 Monitoring Team (NOC) Notifications

Many organisations have a 24/7 monitoring team (NOC) that watches screens round the clock and **phones** the team responsible when something goes wrong. If an outage happens overnight or at the weekend while the team is asleep, the NOC only knows about the monitors that notify it. Site Monitor builds that bridge by e-mail:

- **Opt-in per monitor:** every monitor type and the certificate inventory have a "Notify the 24/7 monitoring team" switch — **off by default**. When it is on, the same alert that produces the team e-mail also e-mails the NOC groups.
- **NOC groups:** e-mail lists defined under **Settings → 24/7 Monitoring Team**, independent of teams (1–50 addresses per group; active / default; a test e-mail button). If a monitor doesn't pick a group, the e-mail goes to the default groups, or to every active group if there is no default.
- **Per-type switches:** if an administrator switches a type off (Ping, say), NOC e-mails stop for EVERY monitor of that type — even monitors that are opted in.
- **Rules:** only alerts at or above the chosen level go out (default: critical only). Team quiet hours, level thresholds and personal mutes don't affect the NOC; maintenance windows and the global e-mail switch do. There is one opening e-mail per alert, and a "RESOLVED" e-mail follows if the opening one went to the NOC (this can be switched off). During a storm or mass outage the NOC, like the team, gets a single combined e-mail.

- **What the e-mail contains:** the subject is `[Site Monitor] [7/24] <LEVEL> — <target> — <Team>`; the body covers what happened, the owning team, the **call list** (name · job title · tappable phone number, read live from Active Directory), the team manager, escalation contacts, the call instructions and a link to the monitor. The mail log files it under the `NOC` category.
- **24/7 monitoring teams (4 October 2026) and operators:** in Settings → 24/7 → "24/7 monitoring teams" one or more teams are selected (global administrators only; the change is audited and a preview shows who will become an operator before saving). Active members of those teams become 24/7 operators **without any role change**: they get **read-only** access to every team's monitors, alerts, incidents, maintenance windows, status page and storms, and can add call logs ( `noc_calls.write` ) and notes to any alert. Acknowledging, resolving, re-notifying, editing monitors or inventory and acting on incidents stay limited to their own teams; the audit log, system health and administration screens stay closed. When a team is removed from the list the access goes away within 30 seconds, without logging in again. The previous set-up keeps working: an **AUDIT** user with `noc_calls.write` is also an operator (the permission is on for AUDIT by default; a permission an administrator deliberately switched off on the **Permissions** screen is left alone).
- **24/7 console:** operators see the organisation's open alerts and recent notifications on the 24/7 tab ( `?tab=noc` ): channels, when it reached 24/7, the last call (who, when, outcome). Filters: team, level, type, "sent to 24/7", called / not called, 1 h / 24 h / 7 days. "Call" opens the owning team's call list (order, name, phone), its manager, escalation contacts and instructions together with the call log form. Alert History also has a "sent to 24/7" filter and a "24/7 · hh:mm" badge, and the alert details' notification log marks the 24/7 email as "delivered to the 24/7 team".

The **24/7 Coverage** screen shows which monitors don't reach the NOC (see §14.31); the warning strip on the dashboard takes you there.

## 9.6 Personal Push Notifications

Personal push sends the alert straight to your phone through the organisation's notification gateway; it is a second channel, completely independent of e-mail. Because the push app is read outside the corporate network, messages are plain text and **contain no links** — open Site Monitor for the details. Administrators manage the channel under **Settings → Webhook Notifications**; you set your own preferences on the **My Activity** page.

**My notification preferences (My Activity).** Next to the "opt out of push" switch and your personal quiet hours:

- **Snooze:** one tap snoozes push for 1 hour, 4 hours or until 08:00 tomorrow (Istanbul time), and "Stop snoozing" switches it back on at once. The card shows the state, for example "Snoozed until 14:30". With **Still send critical alerts** (on by default), only CRITICAL alerts arrive while you are snoozed.
- **Lowest level:** All / High and above / Critical only. Alert pushes below it are not sent to you.
- **Monitor types:** choose which monitor types (certificate, HTTP, DNS…) you want pushes for; at least one must stay selected.
- **Push language:** Turkish or English. The template, the level word and the duration and date formats follow your choice; the alert's own description may stay in Turkish.
- **Send me a test push:** sends a test push to you only, in your language, regardless of your preferences, and shows the result (sent / gateway not configured / HTTP code). At most 3 attempts per 10 minutes.

If you set no preferences, everything works exactly as before. An alert's **resolved** push is not affected by your preferences or a snooze if you received the opening push — the alert on your phone still closes. Sign-in code pushes aren't affected either.

**My push notifications (My Activity).** Lists every push sent to you — and every one that wasn't — over the last 7 or 30 days, grouped by day: the time, the alert (target, type, level, team), the status (Received / Queued / Summarised / Not received), the **reason** if it wasn't received ("Push was snoozed", "Hourly push limit reached", "System maintenance" …) and the text you actually got. The chips at the top summarise the period (received, not received, summarised, the most common reasons) and the All / Received / Not received filter narrows the list. Event-level decisions for alerts of teams you belong to are listed too ("Your team's alert — no push was sent: handed over to the alert storm"); other teams' alerts never appear, and a row for an alert you can no longer see stays as "Another team's alert" without its target. Sign-in code pushes are never stored, for security, so they are not listed. On a phone the list switches to cards.

**Hourly-limit summary.** Notifications over the per-person hourly push limit (30 by default) are no longer lost silently: **15 minutes** after the first overflow (Settings → Webhook Notifications → Hourly limit; 5–120 min) you get one summary push — "Site Monitor: 14 notifications were held back by the hourly limit (3 critical, 11 warning). Latest: site-x - CRITICAL (14:05). Details in Site Monitor." At most one summary per person goes out per interval, and the summary itself is never held back by the limit. While your quiet hours or a snooze are running, the summary waits until they end (it goes at once if it contains a critical alert your rules let through); opting out of push or a system maintenance mute records the summary without sending it. Administrators can switch on **Critical alerts bypass the limit** so that CRITICAL alert pushes are never held back (off by default). In the delivery log, summarised rows link to their summary and the summary lists the rows it covers.

**Escalation step push.** A timed escalation step (see §8.3) also reaches the delayed contact by push, next to the e-mail and webhook, when the contact record is linked to an application user or its e-mail matches exactly one active user. The text reads "[ESCALATION · 15 min unacknowledged] site-x is not responding …", in the person's language. With no match, or when the e-mail matches several users, no push is sent and the reason is written to the delivery log. The person's preferences, the hourly limit, system maintenance and the inactive-account rule all still apply. Administrators can switch it off with **Escalation steps also push to the contact**.

**Administrator view.** The **Notifications** tab in the user details shows the person's push preferences read-only; an administrator can only clear an active snooze (this is audited). **Settings → Webhook Notifications → Templates** has Turkish and English tabs, each with its own preview, plus a separate English title; the test send uses the selected language. The webhook push delivery log labels the new triggers (hourly-limit summary, escalation step), shows skip reasons in plain language and marks English messages with an EN badge.

---

# 10. Monitor Types

Certificate monitoring answers "is the lock sound?"; the types in this section answer "is the door open, and is what is behind it correct?" Each has its own tab under the **Monitoring** group, runs on its own schedule, and belongs to a team — alerts and reports follow that ownership. You add a monitor with the **+ New Monitor** button on the relevant tab, and every form comes with a built-in guide for that type.

Common behaviour: the check interval runs from 30 seconds to 24 hours; an alert requires a number of consecutive failures (the confirmation count, 3 by default) and a recovery is declared after the same number of consecutive successes; you can pause a monitor with the "Active" switch instead of deleting it; and monitors can be clustered with a group label from **Settings → Monitor Groups**. Default intervals, timeouts and slow-response thresholds for new monitor forms come from the `frequency` group under **Settings → General**.

Each type stores its targets and results in its own tables and raises its own alerts. This table is the single answer to "where is this stored, and what will it alert on?":

| Monitor type | Target table | Result table | Alert types raised |
|---|---|---|---|
| Certificate | `certificate_inventory` | `certificate_checks` + `latest_checks` | Expiry, broken chain, revoked, mismatch |
| HTTP / Website | `http_monitors` | `http_checks` | Unreachable, SSL error, domain expiry |
| Port | `port_monitors` | `port_checks` | Port down, slow response |
| DNS | `dns_monitors` | `dns_records` | Resolution failure, record changed, unexpected value, inconsistent propagation, slow query |
| Keyword | `keyword_monitors` | `keyword_results` | Keyword condition failed, slow response, SSL error, domain expiry |
| Ping | `ping_monitors` | `ping_checks` | Unreachable |
| Page integrity | `page_monitors` | `page_checks` + `page_resource_issues` | Page unreachable (critical), page integrity (high) |
| Domain registration | `domain_monitors` | `domain_checks` | Expiry, unknown, status code, registration changed |
| Synthetic (k6) | `scripted_monitors` | `scripted_checks` | Journey failed, slow run |
| Status | `certificate_inventory` (no separate target) | `uptime_checks` | Accessibility |

If you want to switch off alerting for a whole type without deleting the monitors, the `monitoring` group under **Settings → General** has a per-type alert switch that silences notifications while the checks keep running.

**Failure diagnosis and end-to-end diagnosis for every type (5 October 2026).** When a Ping, Port, DNS, Page integrity, PageSpeed, Status, Domain registration or certificate check fails, the reason is recorded too: ICMP not available here, name not resolved (NXDOMAIN / SERVFAIL / no record), connection refused, timeout (firewall), proxy refused or port not allowed by the proxy, TLS handshake / trust / name mismatch, HTTP error status, expected banner not received, broken / timed-out / insecure resources, registry did not answer, and so on. A failed row in the check history shows a reason badge and a one-line explanation; expanding it shows "Why / Impact / What to do", the recorded details (phase, route, resolved IP, response code, timeout …) and the raw error with a copy button. In DNS history a failed lookup no longer looks like "no change": it has its own state and filter. Rows from before this date are shown with the closest reason derived from the error text and carry a "details not recorded" note. Ping, Port, DNS, Page integrity and PageSpeed have **End-to-end diagnosis** in the details header (and on a failed row): for Ping, DNS → ICMP → TCP liveness probes (optional traceroute; tells "ICMP blocked but host alive" apart); for Port, DNS → TCP → TLS / HTTP / banner / UDP, with a direct vs proxy comparison when a proxy is configured; for DNS, every resolver and authoritative name server queried separately (response code, answers, TTL, DNSSEC); for Page integrity and PageSpeed, the same diagnosis as HTTP plus the resource problems or the DNS / connect / TLS / server / download timings next to the thresholds. Diagnosis writes no check row and raises no alert; users who can operate the monitor and hold `diagnostics.run` can run it. Domain, Status and certificate diagnostics are now open to everyone with the same permission and team rights, not only administrators.

## 10.1 Status Monitoring

The **Status** tab gives you an HTTP availability snapshot of the domains in your certificate inventory on one screen: state, response time, uptime percentage and last check time. You can open a diagnostics window per domain to run network, OpenSSL and HSTS tools. Domains inside a maintenance window are excluded from the uptime calculation, so planned downtime does not distort the figure.

## 10.2 HTTP and Website Monitoring

Watches whether a URL is up and returning the status code you expect. You configure the method (GET, HEAD or POST), the expected status (patterns such as `200`, `2xx` or `200-399`; blank means 200–399), redirect following and the request timeout. TLS verification is off by default — you are measuring availability, and internal or self-signed certificates should not get in the way — but switching it on turns an invalid certificate into an alert. You can also add optional SSL and domain expiry reminders.

When should you use it? Whenever you need to know that a service is reachable from outside and answering correctly. You might check your payment API's health endpoint once a minute with `GET /health` and alert your team after three consecutive non-`200` responses — so that you find out before your customers do.

## 10.3 Port Monitoring

For services with no URL — SMTP, databases, bespoke TCP services — monitored by host and port. There are five check types: TCP (is the port open), TLS (does a handshake complete and a certificate get presented), HTTP(S) (status code), BANNER (look for a substring in the response, such as `220` or `SSH-2.0`) and UDP. You can set a slow-response threshold and choose the IP version.

When should you use it? When you need to prove a non-web dependency is alive. Monitoring port 25 on your corporate SMTP server in BANNER mode expecting `220` catches the case a plain TCP check cannot: the port is open but the SMTP service itself has hung.

## 10.4 DNS Monitoring

Watches that a record (A, AAAA, CNAME, MX, TXT or NS) still resolves and that its value has not changed. Queries go out from the monitoring server without caching, and the query timeout is configurable. Three capabilities stand out:

- Expected-value locking: you enter one value per line, and anything in the live answer that is not in your expected set is flagged as possible DNS hijacking. A "pin current value" button fills the list from the live answer in one click.
- Change alerting: when a record value changes, an alert is raised. It does not close automatically and is reminded daily. If every new value is already in your expected list — a known internal-to-external IP switch, for instance — no alert is raised.
- Propagation checking: the record is compared across several public resolvers, and a disagreement between at least two of them is flagged as inconsistent.

When should you use it? Anywhere a domain silently pointing somewhere else would be a catastrophe. Lock the A record of your internet banking domain to its expected IP, and if it flips to an unknown address overnight, the alert is waiting for you before the morning traffic starts.

## 10.5 Keyword Monitoring

Checks whether a specific piece of text appears in a page's body — and how many times. The count condition is flexible: at least, at most, exactly, more than or fewer than. Case sensitivity is optional, and custom HTTP headers plus a `{timestamp}` placeholder in the URL are supported for cache busting. The default interval is one minute.

When should you use it? To catch pages that return HTTP 200 with broken content. Set a condition of "exactly 0" for the text "Under maintenance" on your home page: if the maintenance page is accidentally left in production, you get an alert even though the status code is perfectly healthy.

**Failure diagnosis and end-to-end diagnosis (4 October 2026).** Every check records the final address (masked), the number of redirects, the content type, the size, whether the 2 MB read limit was hit, the charset and the route (proxy / direct). A failed check shows a reason badge and a one-line explanation in the check history (keyword not found, HTTP error status, timeout, DNS, TLS, proxy, read limit …); expanding the row shows "Why / Impact / What to do", the recorded details, likely causes (login page, WAF / block page, page rendered by JavaScript, case / whitespace / HTML entity / charset difference, maintenance or error page, redirected to another site) and a masked excerpt of the page's visible text. **End-to-end diagnosis** in the details header (or "Diagnose this check" on a failed row) runs the same diagnosis as HTTP monitoring: DNS, connection / proxy, TLS, request and response headers, redirects, a curl transcript, a proxy vs direct comparison and a keyword analysis (match count, highlighted contexts, alternative readings). Diagnosis writes no check row and raises no alert; users who can operate the monitor and hold `diagnostics.run` can run it.

## 10.6 Ping Monitoring

Sends ICMP echo packets to a host and measures round-trip time and packet loss. Packet count (1–10, default 4), IP version and timeout are configurable. Some networks block ICMP altogether — for those targets a TCP port check is more reliable — and in a locked-down environment without ICMP permission the result is reported as not applicable rather than as a failure.

When should you use it? When you want pure network reachability, independent of any application layer. Ping your branch router and read a creeping round-trip time on the chart as an early sign of link saturation.

## 10.7 Page Integrity Monitoring

Not about the page itself but about the pieces inside it: every resource extracted from the HTML — images, stylesheets, scripts, iframes, fonts and links — is verified individually over HTTP. There are two modes: single page, and site crawl (depth 0–5, up to 500 pages, with exclusion patterns).

The definition of "broken" is deliberately narrow: 404 and 410, any 5xx other than 503, and a connection that cannot be established at all. Ambiguous codes such as 401, 403, 429 and 503 are recorded as inconclusive and raise no alert, because a WAF or bot filter may well serve them to us while working perfectly in a browser. A failed HEAD is confirmed with a GET and retried once.

A single broken resource marks the page as degraded and opens a high-severity page integrity alert; the alert closes automatically when the resource recovers. Mixed content — an `http://` resource on an HTTPS page — is alerted on by default. Third-party resource alerts are off by default, to keep external CDN noise out of your inbox.

When should you use it? On campaign and shop-front pages, to avoid the embarrassment of "the page loads but the images are broken". Monitor a new campaign page in single-page mode, and when a deployment leaves the main banner returning 404, Site Monitor tells you rather than your customer service desk.

## 10.8 Synthetic Monitoring with k6

For when a single request is not enough — multi-step journeys such as an OIDC sign-in, an API chain or a form login, run end to end with a real k6 script. You start from a template, and the script reads its environment through variables. Secrets never go in the script body: variables marked as secret are stored encrypted, masked in output and logs, and cannot be read back. Run outcomes are classified precisely: a failed k6 check means failure, exceeding the time limit means timeout, other errors mean error, and a run that executed no checks at all is reported as such. If a check could not run at all — the concurrency ceiling was full, or k6 is not installed — the result is skipped: nothing is written to history and no alert is raised, because an infrastructure constraint is not an outage. You can trial a script instantly with "Test Run" before saving it. Use a dedicated service account for monitoring rather than a real user.

k6 is not embedded. Each run executes as a short-lived, sandboxed subprocess on the monitoring server, and its presence and version are verified at start-up and every few minutes thereafter. Outbound traffic respects your corporate network: if a proxy is configured k6 uses it, and your corporate certificate authority bundle is merged with the system roots and handed to k6. The proxy decision is per monitor and has three settings — automatic, always through the proxy, and direct. Note that with the automatic setting, proxy exclusions are matched as suffixes, so an entry for a domain also excludes all of its subdomains.

Every run records more than pass or fail; it stores where the time went, broken down into DNS, TCP, TLS, sending, waiting and receiving, along with the bytes transferred. The "where did it stall?" panel reads that breakdown, so a run hanging in TLS is visibly different from one waiting on a response. A **connection diagnostics** tab probes the same target with and without the proxy, and with and without the corporate certificate authority, to pin down the "Java can reach it but k6 cannot" class of problem. Every saved version of a script is kept, and each run records which version it used, so "it worked yesterday" is an answerable question. If a journey passes but is getting slower, no outage alert will ever fire — for that silent degradation there is a per-monitor **slow run alert** that opens a separate alert when the total duration crosses a threshold, going through the same confirmation and recovery cycle.

When should you use it? When "does sign-in work?" cannot be answered by one HTTP request. Run your online banking sign-in journey — form, OIDC redirect, token, portfolio call — synthetically every five minutes, and when any link in the chain breaks, the k6 output tells you exactly which check failed.

## 10.9 Domain Registration Monitoring

Your SSL certificate can be flawless and you can still lose the domain if the registration lapses. The **Domain** tab checks a domain's registration expiry once a day — expiry dates change rarely, so there is no separate frequency setting. Even if you enter a subdomain, it is reduced to the registrable domain. The warning threshold (30 days by default), the critical threshold (7 days) and the reminder days are configurable per monitor.

The registration tab in the detail window is the domain's identity card: registrar and IANA identifier, the important dates, name servers, resolved A and AAAA addresses with their reverse-DNS names, EPP status codes with explanations, and DNSSEC status. The expiry lookup can be traced step by step with a diagnostic tool, and if a corporate SSL-inspecting proxy breaks the lookup, its certificate chain can be captured and added to the trust store (see §14.26).

When should you use it? On every production domain your organisation owns, without exception. Keep the default thresholds for your main domain and add a 90-day reminder for your brand domains — if renewal needs budget approval, ninety days' notice is exactly what that is for.

---

# 11. Maintenance Windows

A flood of alerts during planned downtime is noise, and worse, it desensitises people to the real thing. The **Maintenance** tab lets you define windows that silence specific monitors — or all of them — for a given period. Operations staff and team administrators use it.

When you define a window you give it a name and description and pick its targets: individual monitors (HTTP, port, keyword, ping, page, DNS, synthetic, domain and inventory certificate domains can be mixed freely) or simply "all monitors". The schedule is defined in a time zone: a start time, a duration in minutes, and a recurrence — one-off, daily, weekly (choose the days) or monthly (choose the day of the month). The recurrence arithmetic is daylight-saving safe.

While a window is active its effect is absolute: no alert is raised for the target monitors and nothing is sent on any channel — not the first alert, not the daily reminder, not even a DNS change. Existing alerts that recover inside the window close silently, with no resolution email either. Checks made during maintenance are excluded from the uptime percentage, so planned downtime does not penalise your availability figures. The active target set is held in memory and refreshed roughly every 30 seconds, and on every change, so the lookup in the sweep path costs nothing.

When should you use one? If you have a database maintenance slot every Sunday between 02:00 and 04:00, define a weekly window: the relevant monitors stay quiet for that period, and at 04:01 normal alerting resumes exactly where it left off.

---

# 12. Security Model

Site Monitor is a monitoring tool, but it is also a corporate record system, which is why authentication, session handling and auditing were part of the design from day one. This section gathers all of the security controls in one place.

## 12.1 Authentication

Authentication is session-based: HTTP sessions are managed by Spring Session and, in the production profile, persisted to the database so that every pod shares them. Local passwords are stored with BCrypt. API requests are guarded by a custom interceptor rather than a full security filter chain.

With directory authentication enabled, sign-in binds to Active Directory, and on the first successful sign-in the user is provisioned automatically along with their organisational role, manager relationship and team. The local bootstrap administrator account always works, as an emergency route in, and the directory and mail settings are reachable only from that account. The bind password is stored encrypted.

"Remember me" gives a persistent seven-day session: a signed token is held in the database and refreshed on every sign-in.

## 12.2 One Active Session

A user holds at most one live session, and the newest sign-in wins. The interface pings the server every few seconds to keep the session fresh. Sign in from another device and the older session gets a 401 on its next request and is redirected to a "session expired" page. Attempting to sign in while a live session exists returns a conflict; accept the confirmation dialogue and the older session is dropped. An administrator can force-terminate any user's session from **System Health**.

## 12.3 Progressive Account Lockout

Lockout hardens in stages against brute-force attempts:

| Failed attempts | Wait |
|---|---|
| 5 failures | 30 seconds |
| 3 further failures | 2 minutes |
| 2 further failures | 10 minutes |
| 1 further failure | 30 minutes |
| Beyond that | Permanent lock, released by an administrator only |

## 12.4 Inactivity Timeout

After a period of inactivity a 60-second warning countdown appears; when it runs out the session is closed automatically.

## 12.5 Container Security

```
securityContext:
  runAsNonRoot: true            # never runs as root
  runAsUser: 1000               # UID 1000
  readOnlyRootFilesystem: true  # root filesystem is read-only
  allowPrivilegeEscalation: false
  capabilities:
    drop: [ALL]                 # all Linux capabilities dropped

# only /tmp and /var/log are writable (emptyDir)
```

## 12.6 The Audit Trail

Every sign-in, sign-out and administrative action is recorded with a UTC timestamp, the user, the client IP, the geographic location, the browser, the event type, the affected resource, the outcome and a field-by-field before-and-after diff. Records are append-only and hash-chained: each row carries its own hash and the hash of the row before it, so tampering with history is detectable rather than silent. Records are archived as line-delimited JSON before the retention job removes them.

Audit entries are enriched with anomaly flags: out-of-hours access, an unusual IP address, an impossible travel velocity, brute-force patterns and rate limiting. The detection windows are managed under **Settings → Login Anomaly**, and an unusual concentration of failed sign-ins can open an incident record automatically.

## 12.7 Monitor Change History

The audit trail belongs to the security team and is open only to admin and AUDIT roles. But the question a team member asks about their own monitor — "who set this up, when, with what settings, and who changed what afterwards?" — is part of the day job. That is why there is a separate, product-facing layer: `monitor_change_log`.

Every monitor, certificate inventory record, monitor group and maintenance window keeps its create, update, delete and rollback events field by field (old → new) alongside a full snapshot of the state. Sensitive fields go through the **same** blocklist as the audit trail and come out masked as `***`. Anyone making a change can add a short reason for it, and the change also lands in the activity feed as a `CONFIG_CHANGED` event.

Getting to it: the **Changes** tab in each monitor's detail view, scoped to the team — another team's history returns a 404 and raises a security event. Administrators also get a **Monitor Changes** console that pages through every monitor's changes in one list. Rolling back to an earlier moment is supported: nothing is erased, the rollback itself

is appended as a `RESTORE` entry, and masked fields and team ownership are deliberately left untouched.

When the feature went live, the monitor events already sitting in the audit trail were carried across once (`AUDIT_BACKFILL`); anything older than the audit trail's own retention window simply isn't there.

## 12.8 Error Handling and Information Leakage

A global exception handler catches everything that is not handled explicitly and returns a helpful message rather than an internal one:

| Condition | HTTP | What the user sees |
|---|---|---|
| Resource not found | 404 | The specific message |
| Conflicting state | 409 | The specific message |
| Invalid argument | 400 | The specific message |
| Not permitted | 403 | The specific message |
| Duplicate record | 409 | "This domain is already in the inventory", and similar |
| Validation failure | 400 | "Invalid field(s): X, Y", with the field list |
| Malformed request body | 400 | "Invalid request format" |
| Missing parameter | 400 | "Missing parameter: X" |
| Anything else | 500 | "Server error" — the stack trace never reaches the browser |

Stack traces and internal error detail go to the log file only.

## 12.9 Masking Sensitive Fields

Full HTTP request and response tracing is available but switched off by default; you enable it by raising the log level for the request logging filter alone. When it is on, roughly sixty sensitive field names across JSON bodies, form bodies and URL query strings are masked automatically:

```
password / passwd / pwd / pass /
old_password / new_password / smtp_pass / db_password /
token / access_token / refresh_token / bearer / csrf /
secret / api_key / client_secret / private_key /
session_id / jsessionid / sid /
pin / otp / mfa_code / verification_code   (English and Turkish variants)
```

Sensitive HTTP headers such as `Authorization`, `Cookie`, `Set-Cookie` and API key headers are masked too. Health checks, favicons, static assets and common static file extensions are skipped entirely, and body logging is truncated at 2,000 characters.

## 12.10 Front-End Error Boundaries

The interface is wrapped in two layers of error boundary: the root layer prevents a blank white screen, and the per-tab layer stops a rendering fault in one tab from taking the rest down with it. The user sees a "something went wrong" message and a refresh button; the full error goes to the console.

## 12.11 Input Validation

Models carry validation annotations and controllers enforce them:

```
@NotBlank @Pattern(...) String domain  // RFC 1123, wildcards permitted
@Min(1) @Max(65535)     Integer port
@Min(1) @Max(4)         Integer tier
```

An invalid body is rejected with a 400 and the list of offending fields, never a stack trace.

### 12.12 Outbound Request Protection

Every monitor points at a URL or host that a user typed, which makes server-side request forgery a real risk. Before any check runs, the target host is resolved and every resolved address is validated; the connection is then made to those addresses rather than resolving again, which closes the DNS rebinding window. For page integrity checks, each parsed resource and each redirect hop goes through the same gate.

The policy has three tiers. Cloud metadata endpoints, multicast and link-local addresses are always blocked, whatever the configuration. Loopback addresses are blocked unless you explicitly allow them. Private and site-local addresses are allowed by default, because this is an internal tool whose whole job is to watch internal services — but an administrator can switch that off live if the deployment is internet-facing.

The same policy is handed to k6 as an address blocklist, because k6 resolves its own DNS and would otherwise bypass the gate entirely.

### 12.13 Identifying the Real Client

Behind a reverse proxy, the client address that matters for auditing and anomaly detection comes from a forwarded header rather than the socket. Site Monitor takes an ordered list of candidate headers and an index into the resulting chain. The index is what makes this safe: a positive index counts from the left, which is only correct behind a proxy that sanitises the header, while a negative index counts from the right, selecting the hop your own infrastructure added, which a client cannot forge. A diagnostic endpoint dumps every known forwarding header alongside the socket address so that operations can pick the right setting rather than guess.

### 12.14 Cross-Origin Requests

Allowed origins are read live on every request, so changing them takes effect without a restart. Two safety rules are built in: a wildcard is stripped from the list, and if the list ends up empty, cross-origin access is disabled rather than opened. Getting the configuration wrong therefore fails closed.

---

## 13. Roles and Permissions

Access control has three layers, and they compose. This section covers each of them, then shows how they combine.

### 13.1 System Roles

| Role | What it is for |
|---|---|
| `ADMIN` | Full access; a global administrator additionally sees the permission matrix, audit log and SQL console |
| `TEAM_ADMIN` | Team administrator, typically the product owner — full control within their own teams |
| `USER` | Everyday user — reads everything in scope, manages their own team's monitors and alerts |
| `AUDIT` | System-wide read-only, for auditors; no settings access |

## 13.2 Organisational Roles

Alongside the system role, each user carries an organisational role — product owner, technical owner, manager or C-level. It does not grant access; it drives escalation routing and weekly report approval.

## 13.3 Team Scope

Users belong to a primary team and may be members of several others. Every inventory entry, monitor, report and alert carries a team, and a user's scope determines which of those records they can see and manage. A blank scope means unrestricted; an empty list means nothing.

## 13.4 Default Permission Matrix

The defaults below can be edited per role from the **Permissions** screen. Administrators are always fully permitted.

| Capability | ADMIN | TEAM_ADMIN | USER | AUDIT |
|---|---|---|---|---|
| Viewing dashboards and certificates | ✓ | ✓ (in scope) | ✓ (own teams) | ✓ (all) |
| Triggering an immediate scan | ✓ | ✗ | ✗ | ✗ |
| Managing the certificate inventory | ✓ | ✓ (in scope) | read only | read only |
| Acknowledging, resending and resolving alerts | ✓ | ✓ (in scope) | ✓ (own teams) | ✗ |
| Logging 24/7 calls ( `noc_calls.write` ) | ✓ (global only) | ✗ | ✗ | ✓ |
| Viewing incidents (Alerts → Incidents) | ✓ (all) | ✓ (own, other teams read only) | ✓ (own, other teams read only) | ✓ (all) |
| Commenting on incidents | ✓ | ✓ (in scope) | ✓ (own teams) | ✗ |
| Managing escalation contacts | ✓ | ✓ (in scope) | read only | read only |
| Editing alert thresholds | ✓ | ✗ | read only | read only |
| Managing teams and users | ✓ | ✓ (in scope) | read only | read only |
| Writing weekly reports | ✓ | ✓ | ✓ (own team) | read only |
| Approving weekly reports | ✓ | ✓ | ✗ | ✗ |
| Configuring monitors | ✓ | ✓ (in scope) | read only | read only |
| Managing maintenance windows | ✓ | ✓ | ✗ | ✗ |
| Viewing the audit log | ✓ (global only) | ✗ | ✗ | ✓ |
| Permission matrix and SQL Playground | ✓ (global only) | ✗ | ✗ | ✗ |
| Directory, mail and general settings | ✓ (bootstrap account only) | ✗ | ✗ | ✗ |

Destructive operations — purging inventory, terminating sessions, force-releasing the scheduler lock — are marked sensitive and ask for confirmation when you grant them. When a new resource key is added, its defaults are seeded on first start-up and backfilled into existing databases on upgrade, with any administrator customisations preserved.

### 13.5 How Access Is Actually Decided

Whether a user can see a screen is the intersection of three layers, not a single flag. Following this order is the quickest way to troubleshoot.

```
1. System role         ADMIN · TEAM_ADMIN · USER · AUDIT
        |              sets the role's default permission set
        v
2. Permission matrix   resource key x action (view / edit / execute)
        |              OVERRIDES the role default - set on the Permissions screen
        v
3. Team scope          view scope  +  manage scope
                       "may you do this" and "may you do this to THIS record"
                       are two different questions
```

Scope sits above permission. A user may hold the inventory edit permission and still only be able to edit their own team's records. A blank scope means unrestricted; an empty list means none — do not confuse the two.

The most visible consequence is the difference between a global and a scoped administrator. An administrator account that arrives from the directory is treated as scoped if it is limited to particular teams: fully capable within those teams, but unable to see the **Permissions**, **SQL Playground** and **Audit Log** screens. Those three are open only to a local administrator with unrestricted scope. The settings screen is narrower still — only the bootstrap administrator account reaches it (see §14.2).

To check your own permissions, open your profile from the user menu; to inspect someone else's, the user record under **Admin Panel → User Management** shows their assigned role and team memberships.

---

## 14. Screens and Actions

This section is a tour of the interface, tab by tab. The tabs are grouped in the sidebar: the Certificate dashboard and the Personal dashboard at the top, then Certificates, Monitoring, Alerts, Reports, Logs and Management. The conceptual detail behind the monitoring tabs lives in 10. Monitor Types; here we focus on what the screens do. We start with the behaviour that is common to every screen — learn it once and it serves you everywhere.

### 14.1 Behaviour Common to Every Screen

The interface is a single-page application, so switching tabs never reloads the page. Even so, where you are lives in the address bar, because the screens share three capabilities.

**Shareable links.** Everything you can see on screen is written into the address: the tab, the team and group filters, the search text, the selected statistic card, the sort order, the page number and any open detail window. Copy an address such as `?tab=keyword&group=Payments&q=api&page=2&monitor=42`, send it to a colleague, and they open precisely the view you were looking at. Every monitoring page and every detail window has a **Copy Link** button that puts the address on the clipboard and confirms with a notification. Default values are never written to the address, so an unfiltered view gives you a clean link. Switching tabs clears the previous page's parameters. Older `?monitor=` links from emails continue to work.

**Pagination.** No list screen ever draws more than two hundred records at once. Every list view — the nine monitoring pages, Status, the certificate cards on the dashboard, the inventory, maintenance windows and the server-side lists — uses the same pagination component. You choose a page size of 25, 50, 100 or 200 (50 by default) and your choice is remembered for that view. The bar underneath shows "Page X of Y · A–B of N records" alongside first, previous, numbered, next and last navigation; past ten pages a box appears so you can jump straight to a page number. Changing a filter or the search text returns you to the first page; the sixty-second auto-refresh never moves you; and a deep link finds its target record whichever page it happens to be on.

**Language, theme and identity.** Click your name at the foot of the sidebar to open the user menu. From there you can see your last sign-in, change your password, and switch the language (English or Turkish) and the theme (light or dark); both are remembered in your browser. If you have permission, **Settings** is there too, with **Report a Problem** just below it, available from every screen.

**Monitoring menu.** The monitor types sit under three sub-headings: **Availability** (HTTP / Website, Ping, Port — is it up?), **Domains and DNS** (DNS, Domain Expiry — is the address right and still valid?) and **Content and experience** (Keyword, Page Integrity, Page Speed, Synthetic — is the page correct, complete and quick?).

Each monitoring page carries two built-in help surfaces: a box at the top of the page explaining what that monitor type does and where its data comes from, and a guide button beside the form showing how to fill the fields in, with examples.

## 14.2 Tab Index and Who Sees What

The table below gives each screen's address-bar key and its visibility rule. Knowing the key is the shortest route to linking someone straight to a screen.

| Group | Screen | Address | Who sees it |
|---|---|---|---|
| — | Certificate dashboard | `?tab=dashboard` | Everyone |
| — | Personal dashboard | `?tab=mydash` | Everyone (widgets follow their own permissions) |
| Certificates | All Certificates | `?tab=all` | Everyone |
| Certificates | Domain Inventory | `?tab=domains` | Everyone; editing depends on permission |
| Certificates | Status | `?tab=uptime` | Everyone |
| Certificates | Expiry Forecast | `?tab=forecast` | Everyone |
| Certificates | Renewal Advice | `?tab=renewal` | Everyone |
| Certificates | Renewal Guide | `?tab=renewal-guide` | Everyone |
| Monitoring | HTTP / Website | `?tab=http` | Everyone |
| Monitoring | Domain | `?tab=domain` | Everyone |
| Monitoring | Port | `?tab=port` | Everyone |

| Group | Screen | Address | Who sees it |
|---|---|---|---|
| Monitoring | DNS | `?tab=dns` | Everyone |
| Monitoring | Keyword | `?tab=keyword` | Everyone |
| Monitoring | Ping | `?tab=ping` | Everyone |
| Monitoring | Page Integrity | `?tab=page` | Everyone |
| Monitoring | Synthetic Monitoring | `?tab=scripted` | Viewing is open; editing needs its own permission |
| Alerts | Warnings | `?tab=warnings` | Everyone |
| Alerts | Incidents | `?tab=incidents` | Everyone — other teams' incidents read only |
| Alerts | Maintenance | `?tab=maintenance` | Everyone; managing depends on permission |
| Alerts | Alert History | `?tab=alerthistory` | Everyone |
| Alerts | 24/7 Coverage | `?tab=noc` | Everyone — monitors within your scope; switching on needs permission to edit the monitor |
| Reports | Statistics | `?tab=stats` | Everyone |
| Reports | Weak Algorithm Report | `?tab=weakalgo` | Permission-based |
| Reports | Weekly Reports | `?tab=weeklyreports` | Permission-based |
| Reports | Incident and Error History | `?tab=incident-history` | Permission-based |
| Logs | Activity Log | `?tab=activity` | Everyone — own teams only |
| Logs | My Activity | `?tab=myactivity` | Everyone — own records only |
| Logs | Audit Log | `?tab=system` | Global administrator or auditor only |
| Management | Admin Panel | `?tab=admin` | Everyone; sub-tabs depend on permission |
| Management | System Health | `?tab=health` | Permission-based |
| Management | Permissions | `?tab=permissions` | Global administrator only |
| Management | SQL Playground | `?tab=sqlplayground` | Global administrator only |
| Management | Issue Reports | `?tab=login-issues` | Permission-based |
| — | Help | `?tab=help` | Everyone |
| — | Settings | `?tab=settings` | Bootstrap administrator account only |

**If you cannot find the Settings screen**, that is why: it is not in the sidebar. It is reached from the user menu at the foot of the sidebar, and only when you are signed in as the bootstrap administrator defined at installation. The restriction is deliberate — the mail password, the directory connection and the encryption key belong to one account.

## 14.3 Sign-In

**Screen:** the application address.

| Action | What happens |
|---|---|
| Username and password | A normal sign-in, local or via the directory |
| "Remember me" | A persistent seven-day session |
| Repeated failures | The counter rises and progressive lockout begins |
| Lockout expires | You may try again |
| Permanent lock | A message is shown; an administrator must release it |
| Report a Problem | A locked-out user can leave a report with a screenshot (see §14.23) |
| Signing in with a session elsewhere | A conflict and a confirmation dialogue; accept and the older session is dropped |

## 14.4 Certificate dashboard

The screen you land on after signing in. An expandable statistics panel — total, valid, warning, error, expiring within 30 days, expired — sits above cards carrying each certificate's domain, days remaining, status badge, issuer and last check.

**For you — today.** The collapsible panel at the top gathers what needs attention across the teams you can see: certificates with under 30 days left, open alerts, flapping, slowing and silent monitors, domain registrations about to lapse, notifications that could not be delivered, certificate health findings and this week's report. Only cards with something in them are drawn; those at zero are listed by name in the single **All clear** line underneath. The **Muted and under maintenance** card shows where the system is deliberately quiet: maintenance windows in progress or starting within 24 hours, paused monitors (with how long they have been paused, shown in the warning colour once a pause passes a week, as it may have been forgotten) and weak-algorithm exceptions that expire within seven days. The ▲ / ▼ marker beside each card's count shows the change on this time yesterday (a rise in red, a fall in green), and the strip under the panel heading sums up the alerts opened and resolved and the certificates renewed in the last 24 hours. The comparison relies on an hourly snapshot, so the markers do not appear on the first day after installation or after a long outage.

| Action | What happens |
|---|---|
| Click a statistic card | Filters to that category |
| Status or expiry filter | Narrows by state or days remaining |
| Search box | Live filter across domain and issuer |
| Sort and page size | Orders by priority or days remaining |
| Click a card | Opens the certificate detail window |
| "Check Now" (if permitted) | Starts an immediate background scan |
| Language and theme | From the user menu, switches between English and Turkish, light and dark |

The **Personal dashboard** ( `?tab=mydash` ) is a second dashboard built from the widgets you choose; the layout is saved to your account and opens the same on every device. **Edit** lets you add, remove, resize and reorder widgets. Clicking a number, tile or row in a widget opens the related screen already filtered to that number (for example the "Critical" tile shows only critical open alarms, an alarm row opens that alarm). The **24/7 widgets** are for the on-duty team: the **Status light** turns red (critical or high alarms open), amber (only warnings) or green (no open alarms) based on the alarms you can see and says how long the oldest one has been open; **Awaiting a call** lists alarms sent to the 24/7 team that nobody has called about yet and turns red after 15 minutes; **Maintenance windows** and **Open incidents** follow the same colour rule. The **24/7 command layout** in the add-widget dialog places these widgets in one click. In edit mode a widget's **Colour panel** button paints the whole tile in the status colour with a big number in the middle; **TV mode** uses this by default (switch it off with **Colour panels** in the top bar). Three Grafana-style grids are available too: the **Service status grid** (each service as a tile in its status colour), the **Monitor type heat map** and the **Team × monitor type** matrix (which team has problems in which type, by cell colour); clicking a cell opens the Monitoring Overview filtered to that team and type. The **Colour panels** group in the add-widget dialog holds single-figure widgets (critical alarms, overdue 24/7 calls, oldest critical alarm, unacknowledged alarms, failing monitors, overdue checks, 24-hour success rate, teams with failures, services in outage, critical incidents and certificate expiries); they are always drawn as colour panels, green at zero and amber or red when there is a problem. The **Colour panel wall** layout places them all at small size in one click. Members of the 24/7 monitoring team see the whole organisation on their Personal dashboard; the chip under the page title states the scope (**Whole organisation (24/7 team)** or **Your teams**). Data that could not be read, or is out of date, is never shown green.

## 14.5 Certificate Detail Window

Clicking any certificate card opens a window with five tabs:

- Details: domain, status, days remaining, subject and issuer, validity window, last check, SAN list and any error message.
- Alerts: every alert this certificate has raised, with acknowledgement and resolution information.
- Notifications: everything that has been sent — recipient, subject, email and webhook status, trigger type.
- Security: full distinguished names, serial number, key and signature algorithms, key usage and extended key usage, chain, revocation and deployment status, OCSP and CRL URLs, SHA-256 fingerprint.
- Notes: free-text notes specific to this certificate, which you can add, edit and delete.

## 14.6 Statistics

Detailed certificate counts, a tier distribution pie chart and a per-team breakdown of valid, warning and error states. Click a tier slice or a team row to filter the **Certificate dashboard** to that slice. Users with a limited scope see only the teams they can reach.

## 14.7 Warnings

Shows only certificates in a warning or error state; filtering and sorting work exactly as on the **Certificate dashboard**. It is the quick worklist for whatever is broken.

## 14.8 All Certificates

A table rather than cards: domain, issuer, subject, expiry date, days remaining, status and last check. The domain, issuer, days remaining and last check columns are sortable.

## 14.9 Renewal Advice

Generates prioritised, plain-language recommendations for each certificate: critical (act now), warning (plan the renewal) and informational (keep an eye on it). It is the weekly agenda for whoever coordinates renewals.

## 14.10 Renewal Guide

Brings your organisation's resources, CA portals and platform notes for renewing certificates together in one place. Most certificates are bought from a commercial certificate authority (CA), so the resources are the heart of the page; generating a key by hand sits folded away at the bottom. From top to bottom:

- **Resources** ( `guide_link` ): internal documents and external links as a grid of cards, grouped by category. The search box filters on title, description, address and category; the category chips (each showing how many links it holds) narrow the list to one category. The order is always CA portals, then platforms (NetScaler, WAF, IIS, Kubernetes / OpenShift and so on), then tools. Each card shows the link type (web page, network folder or email), a title that opens in a new tab, the description, and a short address with a copy button — a browser may not open a network folder, so copy the path and open it in File Explorer. If there are no CA portal links yet, a placeholder heads the list, and an administrator can use it to open "Add the first link" with the category already filled in.
- **Buying or renewing through a CA**: six short steps — order or renew in the CA portal, prove you control the domain (DCV: email, a DNS TXT record or an HTTP file), approval and issue, downloading the right format (PEM with the chain, or PFX), installing it on the endpoints, and checking it in Site Monitor. The details sit behind the information button next to each step, and the steps take you to the matching resource category, the platform notes, or All Certificates and Renewal Advice.
- **Installing on your platform** (collapsible): short notes and copyable commands for NetScaler, IIS, Kubernetes / OpenShift and Nginx / Apache, plus a button to that platform's resource category when there is one. Your chosen platform is remembered in your browser.
- **Advanced: generating your own key and CSR** (collapsible, closed to begin with): the manual route you'll rarely need — scope, key and CSR, matching the certificate to the key and building the chain, installing, checking and wrapping up. You can tick steps off as done; the ticks are only saved in this browser, and the heading shows your progress while the section is closed.

The "On this page" row in the header jumps to each section (opening the collapsible ones); it isn't shown on phones. The header also has buttons for Renewal Advice and Domain Inventory, and administrators get **Add Link**. An administrator adds links, edits or deletes them from the card's Actions menu, and orders them by number.

## 14.11 Expiry Forecast

The analytical view of certificate expiry: headline cards (critical at 7 days or fewer, high at 8–14, warning at 15–30, total active), a daily expiry distribution chart, a calendar heat map, load distribution by team and status, and a list of what is coming up. Click a day in the calendar to see the certificates expiring on it. This is the capacity planning screen — it answers "how many renewals are stacking up in November?" at a glance.

## 14.12 Domain Inventory

The register of domains to be monitored. The table shows domain and port, tier badge, team, owner, description, active state and the available actions — edit, transfer and delete. A global administrator manages everything; a team administrator manages the teams they lead; managers and read-only roles can look but not touch.

Adding a domain means filling in a five-part form:

- Basics: the fully qualified domain name, the port (443 by default), the owning team, the tier, the owner and whether it is active.

- Operational flags: external vendor, action required, OpenShift, SSL pinning, internal certificate, JKS keystore, server update, load balancer, WAF enabled, in use, EV certificate, check through the proxy. Apart from the proxy flag these do not change the check pipeline; they are operational metadata that shows up in reports and filters.
- Process information: who purchased it, and for which business unit.
- Descriptions: a general description and a process note.
- Advanced: the expected SHA-256 fingerprint and the expected subject, used for deployment compliance checking.

Deletion is permanent and cannot be undone: the confirmation names the records and what goes with them, and the card leaves the list as soon as you confirm. Transfer moves a certificate to another team. Rename a domain and its entire history moves atomically with it (see §3.9).

## 14.13 Weak Algorithm Report

Lists certificates using insecure algorithms such as SHA-1 signatures or 1024-bit RSA keys: domain, signature and key algorithm, key size, weakness type, criticality, owner, team and expiry date. It is the first screen the security team opens during a compliance sweep.

## 14.14 The Monitoring Tabs

**Status**, **HTTP / Website**, **Port**, **DNS**, **Keyword**, **Ping**, **Page Integrity**, **Synthetic Monitoring** and **Domain** all follow the same layout, so learning one teaches you all of them.

The top of the page carries an explanation box for that monitor type and a statistics strip — total, healthy, failing, paused — where clicking a card filters the list to that state. Below it sits a paginated monitor list you can narrow by status, group, team and free text. Top right you will find **+ New Monitor** and a **Guide** button for that type.

In HTTP / Website, Port, Ping, DNS, Keyword, Page Integrity, Page Speed and Synthetic monitoring, each monitor card shows the last 24 hours' response-time trend and the last five checks, with a 30-day availability line at the bottom (for example `30 d: 99.95% · target 99.9% ✓`). The **1 day · 7 days · 15 days · 30 days** labels on the right of that line tell you whether anything failed in each period: a clean period is green, a period with failures carries a ⚠ icon in amber, and one whose availability fell below the target carries a ⚠ icon in red; a period with no checks at all is greyed out. Hover over a label to see that period's number of checks, failures and availability, followed by every hourly slot in which something failed and how many failures it saw (for example `23/09 17:00–18:00 — 5 failures`); the six most recent slots are listed and any others are counted underneath. On a Port card the monitored endpoint sits just under the host name: the `:port` in large type, a protocol badge coloured by check type (TCP, TLS, HTTP, Banner, UDP), the path being checked for the HTTP type and, for well-known ports, the service name (for example 5432 → PostgreSQL). A Ping card shows, in the same place, a coloured **ICMP** badge (**ICMPv6** for IPv6), the IP version if one was chosen (IPv4 or IPv6) and the number of packets sent per check.

**Port monitoring through the proxy.** The **Corporate proxy** field on the Port form is **Direct** by default, so existing monitors keep their route. Choose **Always via proxy** or **Same as inventory** and TCP, TLS, HTTP and Banner checks run through the corporate proxy `CONNECT` tunnel; the result then shows the target is reachable *from the proxy network*, and the response time includes the proxy hop. Proxies only open tunnels to the ports they allow, so the form always lists the **ports that can be checked through the proxy** (443 and 8443 by default; see Settings → Monitoring → *Ports the proxy allows CONNECT to*). With the proxy selected, the form warns you if the port is not on that list or if the type is **UDP**: UDP cannot go through an HTTP proxy and always runs directly. If the proxy refuses a tunnel, the check fails with **"proxy refused the tunnel"** (with the allowed ports in the message) rather than reporting the port as closed.

Clicking a monitor opens a tabbed detail window:

- **Check history** — paginated, with the outcome, duration and any error message on each row.
- **Alert history** — the alerts this monitor has raised, with acknowledgement and resolution details.
- **Response chart** — preset ranges of 24 hours, 7 days, 30 days and 90 days plus a custom range; the average, the minimum–maximum band and the 95th percentile are drawn together, with outage periods shaded red. On a ping chart, packet loss appears on a second axis. The default range is 24 hours.
- **Notes** — a free-text notebook for that monitor.

The detail window also offers **Check Now** (if you are permitted), **Copy Link**, edit, duplicate, and pause or delete. To silence a monitor temporarily without deleting it, turn off its "Active" switch.

For the behaviour behind each type, see 10. Monitor Types.

## 14.15 Incidents

The worklist of outage events derived from monitoring alerts. Each row shows the start time, status, severity and root-cause class; you can filter, comment and close resolved events. "How long did it last and when did it close?" is answered here.

The **My teams' incidents · Other teams' incidents · All** filter at the top (with counts) also shows incidents raised for other teams; the default view is your own teams only. Another team's incident is **read only**: an "Another team's record — read only" badge and the owning team are shown and you can read its comments, but acknowledging, resolving, commenting and deleting are left to that team (the server refuses them as well). Notification recipients, delivery logs and 24/7 call logs stay with the owning team. A global administrator can switch this visibility off with `site.monitor.incidents.visible-to-all` under **Settings → General** (on by default); global administrators and AUDIT users already see every incident, so the filter isn't shown to them. When you switch filters the results stay where they are; if the response is slow the list dims and the new results replace it once they arrive.

## 14.16 Maintenance

Where you manage the windows that suppress alerting during planned downtime; the mechanism itself is described in 11. Maintenance Windows. The screen lists defined windows with their name, target count, time zone, start, duration, recurrence and whether they are currently active. You create, edit, temporarily deactivate or delete them. Team administrators manage their own teams' windows; a global administrator sees them all.

Upcoming windows are gathered at the top of the list, so "which monitors go quiet tonight?" takes one glance.

## 14.17 Alert History

The archive of every alert and the place you act on them. All twenty-eight alert types the product raises are recognised here and shown with a readable label.

The filter bar at the top searches on the server, so it stays fast across hundreds of thousands of records: free-text search, alert level, alert type, team, ownership (acknowledged or not) and an open-alerts-only switch. A statistics strip immediately below shows the distribution of the selected set. Copy the link to share the filtered view.

You can collapse records into groups by subject, which folds dozens of alerts from the same cause into a single row. Grouping and the statistics strip are off by default, keeping the plain list view. Each row carries badges showing **how long** the alert has been open and how many times it has **recurred**. An **Export CSV** button downloads the filtered set as a table.

| Action | What it does |
|---|---|
| Acknowledge | Marks it as seen; daily reminders stop, the alert stays open |

| Action | What it does |
|---|---|
| Resend | A manually triggered notification to every escalation contact, with no quota applied. You can review the recipient list and remove individuals before sending |
| Mark resolved | Closes the alert, records who resolved it and sends a resolution notice |

> **Acknowledging and resolving now require a reason.** Both actions ask you for a short explanation, and it is mandatory. The reasoning is simple: six months later, whoever reads the alert history is not asking who closed it, they are asking **why**. The note is stored on the alert and shown when the row is expanded.

Expanding a row reveals the notification history — recipient, subject, email and webhook status, trigger type — along with the acknowledgement and resolution details and their notes.

## 14.18 Weekly Reports

Where teams write their weekly operations report and the product owner approves it. You pick the week and fill in the incident summary (counts of urgent and high-severity events), the list of open problems and the planned work; a status badge shows draft, submitted, approved or returned.

The flow: submit the report for approval, and the product owner receives an HTML email with an approval link. They can approve or return it from the screen or straight from the email — the token link deliberately works without signing in. On Friday morning a reminder goes out for anything still pending, and a report can be handed to another team. Everyday users write their own team's report; product owners see and approve the teams they lead; a global administrator sees everything.

## 14.19 Incident and Error History

The SRE incident ledger, written by hand rather than derived from monitoring. Each record carries the title, when it happened, when it was detected and resolved, severity, status, category, error and channel codes, the affected service and application, a root-cause summary, the resolution steps, the business impact and whether the service level agreement was breached. Screenshots can be attached. Summary tiles at the top of the screen break the set down by status and by whether the SLA was met.

## 14.20 Activity Log

The operational stream of what the monitors actually did: which monitor ran, against which target, what the outcome was, how long it took and any error. It is isolated by team on the server, so you only ever see your own teams' activity — including in the detail view.

## 14.21 My Activity

Your own record: the actions you have taken and the sign-ins recorded against your account. It exists so that everyone can answer "what did I change and when?" without needing audit access. The same page holds your push preferences (snooze, level, monitor types, push language, self test) and your **My push notifications** history — see §9.6.

## 14.22 Audit Log

The security and compliance trail, open to global administrators and auditors only. It shows the timestamp, the actor, the client address with its geographic location, the event type, the affected resource, the outcome and the before-and-after diff, alongside any anomaly flags. Records are append-only and hash-chained.

## 14.23 Issue Reports

Where reports submitted through **Report a Problem** land, including those left by users who could not sign in. Each report carries the description, screenshots, the browser and the client address. Reports can be acknowledged and resolved, and a daily digest can be emailed.

**Error messages and request id (8 October 2026).** Every error in the application says what happened, the likely cause and what to do next; network loss, timeouts and missing permissions are each explained on their own. The **Technical details** section under an error shows the HTTP status, the error code and the **request id**; copy that id into your report and the administrators can find the entry in the server log at once. Going to an address that does not exist opens Site Monitor's own 404 page, with links back to the home page and the most used pages.

## 14.24 Admin Panel

The everyday administration screen, divided into sub-tabs:

- Alert thresholds — the day boundaries for warning, high and critical, and the reminder interval (administrators only).
- Escalation contacts — who receives alerts for which team, from which level, with an optional webhook.
- Team management — teams, their leaders, their email addresses and their reporting switches.
- User management — accounts, roles, team memberships, password resets and lock releases.

## 14.25 Permissions

The permission matrix: resource keys down the side, actions across the top, roles as the editable dimension. Sensitive actions ask for confirmation when granted. Open to global administrators only.

## 14.26 Settings

The application-wide configuration screen, reachable only by the bootstrap administrator account, from the user menu. Its sections:

- General — the application base address, the system administrator email, allowed cross-origin sources, and the problem-reporting switches.
- Branding — the application name, tab title, sign-in screen text, primary colour, logo and the announcement banner.
- Monitor groups — the group labels used to cluster monitors.
- SMTP — mail server settings, with a test message button.
- Weekly availability — the Monday email, its history, a preview and the outage PDF.
- Certificate inventory report — the scheduled monthly inventory report and its recipients.
- Storm — alert storm thresholds and behaviour.
- 24/7 Monitoring Team — NOC groups (multiple e-mail addresses, active/default, test e-mail), per-type switches, minimum level, the resolved e-mail and call instructions; a scoped administrator sees it read-only, with the addresses hidden.
- Login anomaly — failed sign-in detection rules and recipients.
- Directory — the LDAP connection, a test bind and an attribute viewer.
- Domain diagnostics — a step-by-step trace of a registration lookup, and the tool that captures a proxy certificate chain as paste-ready text.
- Retention — the retention period for each table, a dry run, the run history and the legal hold switch.
- Database — schema information, table sizes and a schema diagram.

- Secrets — the state of the encryption key and the tools that depend on it.

### 14.27 System Health

The operational dashboard for the platform itself: heartbeat history, mail delivery statistics, database metrics, scan staleness and the scheduler lock. It also carries the database analytics — the busiest and slowest queries, table sizes and connection counts — and the user and session view with sign-in trends, top users and sources, an anomaly breakdown and a weekly activity heat map. Administrators can force-release a stuck scheduler lock and terminate another user's session from here.

**Metrics** (the "Open metrics" card at the top of the page): Site Monitor's own measurements, per pod. Every pod writes one sample per minute and samples are kept for 7 days (Settings → General, "System metrics retention"). At the top a summary of the current values (live pods, highest CPU and memory, database pool, task queue, last hour's HTTP requests and ERROR log lines), below it the pod list (live; silent = a pod that went quiet without a shutdown marker, a warning; stopped = a pod that shut down cleanly in a rollout or scale-down, not a warning; last sample, uptime) and the charts: CPU, heap, threads, GC pause; requests per minute, error rate, response time; database connections and waiting requests; task pool, queue and overflow; log lines and cache hit rate. The range is 1 hour / 6 hours / 24 hours / 7 days and the pod picker shows a single pod; the choice is written to the address bar so you can share the link. A value that cannot be read shows as "—" and never counts as green; a slice with no sample from a pod stays a gap in the chart.

### 14.28 SQL Playground

A read-mostly SQL console for global administrators: the table list, columns and relationships, sample queries and a query history. Every execution is audited.

### 14.29 Help

The screen you are reading. It renders this guide with a table of contents, scroll tracking and a **Download PDF** button. Its language follows the application language, and the PDF you download matches it.

---

### 14.30 Product Tour

A user signing in for the **first time** sees a welcome card shortly after the data loads: "Welcome to Site Monitor — a two-minute tour?". There are three choices:

- **Start the tour** — a 17–20 stop walk-through (depending on your role) that spotlights the main places from the left-hand menu to the help button in the bottom-right corner.
- **Not now** — the card closes; it asks again on at most three further sign-ins, then stays quiet.
- **Don't show again** — permanent and kept **on the server**: the card never comes back, even from another browser or computer.

**During the tour** the screen is dimmed and only the item being explained stays lit and clickable. The balloon carries a title, a short explanation, progress (e.g. 7/20), **Back / Next**, "Read more" (opens that topic's guide section in the side panel), "Skip the tour" and "Don't show again". Keyboard: → next, ← back, **Esc** closes. Some stops ask you to do something ("click the card", "press Ctrl K"); the tour moves on when you do, or **Do it for me** does it on your behalf.

**Content by role:** everyone sees the dashboard counters, filters, "Check Now", the certificate card and detail window, All Certificates, Monitoring, Alerts, Reports, the command palette, notifications, the user menu (including theme and language) and help. Team admins also see the add-domain stop; admins see Admin and System Health; the auditor (AUDIT) role sees the Audit Log stop. On a narrow screen (phone) a shortened version opens as a bottom sheet.

**Page tours:** some pages carry their own short tour (All Certificates: filters, status menu, columns, presets, CSV, row selection, row menu; HTTP monitoring: how it works, new monitor, how to fill in, cards). The first time you land on such a page a chip appears in the bottom-right corner: "Want a quick look around this page?". Close it and it will not return for that page.

**Getting-started list:** at the top of the dashboard a small six-item list guides a new user (finish the tour, open a card, browse All Certificates, look at a monitoring page, see the weekly reports, open the help guide). Items tick themselves off as you visit the pages; the list disappears once complete or when you hide it with ×.

**Finding the tour again:** user menu → **Product tour**; the "?" help drawer in the bottom-right corner offers **Product tour** and, where one exists, **Tour this page**; the button on the Help page; the command palette (Ctrl K) command "Start the product tour". Even after "Don't show again" you can open the tour this way whenever you like.

**What's-new tour:** when a release adds new stops, users who already finished the tour see a short "What's new" card at sign-in covering only the new stops; it, too, can be closed with "Not now" or "Don't show again".

**For administrators:** the KPI under System Health → Users shows how many users completed, dismissed or never saw the tour. **Reset the tour** in the user edit window clears a person's tour state so they see the welcome card at their next sign-in. Completion, dismissal and reset are written to the audit log (`TOUR_COMPLETED`, `TOUR_DISMISSED`, `USER_TOUR_RESET`).

### 14.31 24/7 Coverage

If something breaks overnight your team may well be asleep, and the 24/7 monitoring team only hears about monitors that are set to notify it (see §9.5). This screen (`?tab=noc`) shows which monitors within your scope **don't** reach the 24/7 team, and why:

| Part | What it shows |
|---|---|
| Summary | Total / covered / not covered / paused tiles (click to filter), the coverage percentage, and coverage bars per type |
| Reasons | Monitor not opted in · Type disabled by admin · No active 24/7 group · Paused |
| Filters | Search, team, type and reason; shareable address parameters with the `n_` prefix |
| Actions | **Notify the 24/7 team** in one click on monitors you can edit; bulk selection with "Notify the 24/7 team (N)"; switching it off from the "…" menu on an opted-in monitor |
| Call list | An ordered list of people per team — the team manager, a team administrator or an administrator edits it (move up/down, add/remove). Anyone with no phone number in Active Directory is flagged; if the list is empty, the e-mail shows the team manager instead |

If there is no active group, an administrator sees a link to Settings and everyone else sees a note to ask their administrator. The dashboard strip "N monitors aren't set to alert the 24/7 monitoring team if they go down overnight" leads here and can be hidden for the session.

---

## 15. Operational Procedures

This section is the recipe book for everyday work: the tasks people do often, step by step, with the screen names.

## 15.1 Adding a Domain

1. Go to the **Domain Inventory** tab and click add.
2. Under basics, enter the domain (for example `api.example.com`), the port (usually `443`), the owning team and the tier.
3. Tick the operational flags that apply and save.
4. The system checks the domain on the next hourly sweep; the first results appear on the **Certificate dashboard**.

Tip: use "Check Now" if you do not want to wait for the sweep.

## 15.2 When You Receive an Alert

1. Read the domain and days remaining from the email.
2. If you know what is going on, acknowledge the alert on the **Alert History** screen with a short note — the daily reminders stop.
3. Start the renewal process with the CA, the team and the platform.
4. When the renewal is complete, the system picks up the new certificate on the next sweep.
5. Once it is genuinely fixed, mark it resolved with a note; a resolution notice goes out.

For a critical alert: acknowledge it straight away, start the renewal immediately, use resend if you need management informed at once, and mark it resolved when the work is done.

## 15.3 After a Certificate Is Renewed

1. Deploy the new certificate to the server or platform.
2. Site Monitor detects it on the next hourly sweep.
3. If you have recorded an expected fingerprint in the inventory, deployment compliance is verified too.
4. Mark the relevant alert resolved on the **Alert History** screen; the resolution notice is sent automatically.

## 15.4 Setting Up a Team

1. Create the users: **Admin Panel** → user management, with the appropriate system and organisational roles.
2. Create the team: team management, with its leader and team email address.
3. Assign users to the team from the user edit window.
4. Define escalation contacts: a minimum alert level for each person, and a Teams or Slack webhook if you want one.
5. Assign certificates to the team: **Domain Inventory** → edit → owning team.

If you use the directory, steps 1 to 3 largely happen by themselves: the user is provisioned on first sign-in and their manager relationship is picked up automatically.

## 15.5 An Escalation Contact Template

```
Scenario: escalation for one team

Product owner:     minimum level WARNING  → told about everything
Technical owner:   minimum level WARNING  → told about everything
Manager:           minimum level HIGH     → high and critical only
Director:          minimum level CRITICAL → critical only
```

## 15.6 Planning a Maintenance Window

1. Create a new window on the **Maintenance** tab with a name and description.

2. Choose the targets: the affected monitors, or "all monitors".
3. Set the time zone, start time, duration and recurrence — one-off, daily, weekly or monthly.
4. While the window is active no alerts are raised, nothing is sent and the uptime percentage is unaffected; when it ends, monitoring resumes exactly where it left off.

## 15.7 When the Application Has Been Down

While the application is down, scanning stops and nothing is sent. On restart, catch-up takes over: any of that day's outstanding daily notifications are sent immediately, without a network call, and the full sweep follows. However long the outage lasted, the day's notifications still go out.

## 15.8 Automatic Clean-Up of Old Records

Every check writes a row to the database; five hundred monitors running once a minute produce half a million rows a day. The retention policy is what keeps that in check. You manage it from **Settings → Retention**, where each row represents a table and its retention period in days.

Clean-up runs as a single job at 03:00 every night, and the order matters:

1. **Roll up first.** Raw check rows are summarised into the daily and hourly rollup tables, looking a few days back with an idempotent upsert. The long-term trend survives even after the raw data goes.
2. **Then delete.** High-volume tables are purged in batches (10,000 rows by default), keeping each transaction short so locks are not held.
3. **Finally analyse.** Statistics are refreshed so the query planner works against the new table sizes.

Notable defaults:

| Record | Default retention |
|---|---|
| Raw check series (status, certificate, port, keyword, ping, DNS, HTTP, domain) | 180 days |
| Activity log | 365 days |
| Audit log | 365 days, archived as JSON before deletion |
| Notification log | 365 days |
| Daily rollup / hourly rollup | 730 days / 365 days |
| Per-minute HTTP metrics | 7 days |
| Heartbeat | 30 days |
| Page resource issues | 90 days |
| Images (incidents, weekly reports) | 730 days |
| Incident records | 0 = never deleted (opt in if you want it) |

Three behaviours are worth knowing. **Only closed records are deleted** — an open alert, an unresolved incident or an ongoing storm survives regardless of age. **DNS and domain series always keep a baseline row per monitor**, because without it there is no reference left for the "has this changed?" comparison. And while the **legal hold** switch is on, nothing is deleted from anywhere — during an audit period you can stop all clean-up with a single toggle.

Every run is written to the retention history, so the **Settings → Retention** screen tells you which night removed how many rows from which table. Before changing a policy, use **Dry Run** to see how many rows would be affected.

### 15.9 Using a Proxy in Production

Some targets have a WAF or firewall that rejects the monitoring pod's address. For such a domain:

1. Put the proxy address into the production values file for your environment and run a Helm upgrade. The values reach the ConfigMap, and the configuration checksum annotation rolls the pod for you.
2. Edit the domain on the **Domain Inventory** screen and turn on "check through the proxy".
3. On the next sweep only that domain's check goes through the proxy; the tunnel steps are visible in the logs.

> **Do not pass operational values with command-line overrides.** Values supplied only on the command line live in that release's own values and disappear silently when the release is renamed or reinstalled. This has actually happened: proxy settings were lost during a release migration and domain registration lookups timed out for days. Anything production depends on belongs in the environment values file.

A warning: routing all pod traffic through the proxy will break the checks that currently work — flag only the domains that have a problem. List any internal addresses you want to keep direct in the proxy exclusion list; matching is by suffix, so an entry for a domain also excludes all of its subdomains.

---

## 16. Deployment and DevOps

Site Monitor ships as a single container image — the React interface is served as static content from inside the back-end jar, so there is no separate web server. This section covers bringing the application up from scratch, how the image is built, and how releases flow.

### 16.1 Local Development

You will need **Java 25**, **Maven 3.9 or later** (there is no Maven wrapper in the repository, so Maven must be installed), **Node.js 20 or later**, and a running PostgreSQL.

Start with the configuration file:

```
cp .env.example .env      # database, mail, admin credentials
```

Spring Boot does not read `.env` itself — Docker Compose and the local start-up script read it and translate the values into system properties. If you run the back end directly, supply the values as environment variables.

Back end, from the `backend` directory:

```
mvn spring-boot:run          # development server on :8080
mvn -B clean verify          # what CI runs: tests plus coverage
mvn package -DskipTests      # produces the deployable jar
```

Front end, from the `frontend` directory:

```
npm install
npm run dev                  # Vite on :5173 — /api and /metrics proxy to :8080
npm run build                # produces dist/, with the version baked in from VERSION
npm run test                 # unit tests
npm run test:coverage        # with the coverage thresholds enforced
npm run test:e2e             # Playwright; starts its own Vite on :5174
```

The two ports are deliberately separate: the end-to-end tests use 5174 so that they never kill the development server you have open on 5173.

On Windows there is a script that brings the whole stack up in one go: it reads `.env`, stops only the process listening on 8080 (not every Java process), starts the newest jar and waits for the health endpoint to report healthy.

> After changing back-end code, do not restart without repackaging. The start-up script runs the compiled jar, and running the tests does not rebuild it — otherwise the application carries on serving the old code and your new fields come back empty.

## 16.2 The Container Image

```
Registry : ghcr.io/<owner>/site-monitor
Tags:
  latest               (current production)
  vX.Y.Z               (release tag)
  develop-a1b2c3d      (development commit)
  X.Y.Z-rc / staging   (release candidate)
```

The image is built in four stages:

```
grafana/k6            →  the k6 binary is copied out (for synthetic monitoring)
node:20-alpine        →  npm ci + vite build  →  dist/
maven:3.9-temurin-25  →  mvn package          →  app.jar
temurin-25-jre-alpine →  runtime
```

The runtime layer is deliberately well-equipped: `curl`, `bash`, `dig`, `openssl`, `traceroute` and the ping utilities are installed, because the in-application network diagnostics and the ping monitor use them. The container runs as a non-root user with a read-only root filesystem and all capabilities dropped; the only writable paths are `/tmp` and `/var/log`.

For a local full stack:

```
docker-compose up -d          # application on :8080
```

## 16.3 Installing with Helm

The chart produces the following templates:

| Template | What it does |
|---|---|
| `deployment.yaml` | The application pod — probes, security context, volumes, downward-API environment |
| `service.yaml` | ClusterIP, 80 → 8080 |
| `ingress.yaml` | Host, TLS secret, nginx annotations |
| `configmap.yaml` | All non-secret configuration, injected as environment variables |
| `secret.yaml` | Passwords and the encryption key, retained on uninstall |
| `serviceaccount.yaml` · `namespace.yaml` | Identity and namespace |
| `hpa.yaml` · `pdb.yaml` | Autoscaling and disruption budget — switched off in the single-pod configuration |

**There is no CronJob.** Every scheduled job runs inside the application process, under a database-backed distributed lock. That is what guarantees the same sweep never runs twice, even if you scale to several replicas.

Installing:

```
helm upgrade --install site-monitor ./helm/site-monitor \
  -f helm/site-monitor/values.yaml \
  -f helm/site-monitor/environments/master.yaml \
  --set image.tag=$(cat VERSION) \
  --set secret.adminPassword=$ADMIN_PASSWORD \
  --set secret.dbPassword=$DB_PASSWORD \
  -n site-monitor --create-namespace
```

Or straight from the published chart:

```
helm install site-monitor oci://ghcr.io/<owner>/sitemonitor-chart --version <X.Y.Z>
```

## 16.4 Where the Values Come From

Helm values are applied in layers, and **each layer overrides the one above it**:

```
helm/site-monitor/values.yaml            <- chart default (single-pod profile)
        |
        v  added with -f
helm/site-monitor/environments/<env>.yaml    <- environment profile
        |                                       develop . release . master
        v  added with --set
command line                             <- SECRETS AND IMAGE TAG ONLY
```

| Values file | Environment | Notes |
|---|---|---|
| `develop.yaml` | Development | Email off, infrequent scanning, one replica |
| `release.yaml` | Staging | A production-like verification environment |
| `master.yaml` | Production | Ingress host, proxy settings, database pool, allowed origins |

> Command-line overrides are for **secrets and the image tag only**. Persistent values that production depends on — the proxy address, the ingress host, the pool size — must live in the environment values file. See the warning in §15.9.

Configuration reaches the pod as **environment variables**: the ConfigMap carries the non-secrets while the Secret carries the administrator password, the database password, the mail password, the encryption key and any proxy credentials. Configuration checksum annotations on the deployment mean the pod rolls itself when configuration changes — no manual restart needed.

If the application base address is left blank, the chart derives it from the ingress host and TLS setting; links in emails use that address.

### 16.5 Versioning and Release Flow

```
commit (conventional prefix)
        |
        v
   ci.yml  -- 4 jobs --+- backend      : mvn -B clean verify
                       +- frontend     : npm ci -> test:coverage -> build -> audit
                       +- frontend-e2e : Playwright / Chromium
                       +- helm-lint    : against all three environment values files
        |
        v (main branch only)
  release.yml
        +- Gate    : is CI green for this commit? if not, no release
        +- Detect  : feat -> minor . fix/chore/refactor -> patch . breaking -> major
        +- Write   : VERSION + Chart.yaml (version and appVersion)
        +- Image   : buildx -> registry  +  vulnerability scan
        +- Chart   : helm package -> OCI push
        +- Tag     : vX.Y.Z  -> GitHub Release
        +- Back-merge: main -> develop
```

A fourth workflow scans dependencies against the vulnerability database every Monday. It is kept off the release path because it is slow.

Two rules protect the release pipeline:

- **Never edit the version files by hand.** The release workflow writes both; a manual bump produces a conflict on the next rebase. You commit code with a conventional prefix and let CI decide the version.
- **Never put a CI-skip token in a commit body.** The platform skips every workflow for that push, including the release gate.

The single source of the version number is the root `VERSION` file. The interface bakes it in at build time and the back end reads it at runtime. The version fields inside the build files are deliberately left alone — nothing reads them.

---

## 17. Resilience and Availability

If the monitoring platform goes down, so does your alerting. Site Monitor runs as a single pod in production, so resilience rests on two things: the pod coming back quickly and in the right order, and no state ever living in the pod's memory. This section covers those mechanisms, and ends with what is already waiting if you want to scale out.

### 17.1 Update Behaviour

```
replicaCount: 1
strategy:
  type: RollingUpdate
  maxSurge: 1          # the new pod comes up first
  maxUnavailable: 0    # the old one only goes when the new one is ready
```

On a single-pod deployment `maxUnavailable: 0` is the setting that matters most: the old pod carries traffic until the new one passes its readiness probe, so an upgrade is invisible to users. A generous termination grace period, paired with graceful shutdown in the application, means in-flight requests and a running sweep are not cut off mid-way.

## 17.2 Health Checks

| Type | Endpoint | Delay | Period | Failure threshold |
|---|---|---|---|---|
| Startup | `/health/readiness` | — | 10s | 30 → restart (about a 300-second budget) |
| Readiness | `/health/readiness` | 5s | 5s | 3 → traffic withdrawn |
| Liveness | `/health/liveness` | 0s | 15s | 6 → restart |

Each of those numbers has a reason. **The start-up budget is generous** because the first boot after an upgrade validates the schema, applies patches and warms the settings caches; a tight budget puts the pod into a restart loop. **Liveness does not look at the database** — a brief database outage does not make the application itself unhealthy, and restarting the pod would only make things worse. **Readiness refuses traffic during start-up**, so the pod never looks ready while leaving the first requests hanging.

## 17.3 Where State Lives

Nothing durable lives in the pod's memory, so a restart loses nothing.

| State | Where it lives | On pod death |
|---|---|---|
| User sessions | The session table in the database | Preserved — nobody has to sign in again |
| Scheduler lock | The lock table, with a TTL | Released when the TTL expires |
| Alert state and confirmation counters | The alert tables | Preserved |
| Maintenance window cache | In memory, refreshed every 30 seconds | Rebuilt |
| Settings caches | In memory, refreshed every 10 seconds | Rebuilt |

## 17.4 Catching Up After Downtime

While the pod is down, scanning stops and nothing is sent. On start-up, catch-up runs: any of that day's outstanding daily notifications go out immediately without a network call, and the full sweep follows. However long the application was down, the day's notifications are still delivered.

Heavy start-up work is deferred by a configurable delay, so that in the first seconds after boot the CPU goes to users trying to sign in and to password verification.

## 17.5 Moving to Horizontal Scaling

If one pod is not enough, the chart is ready: turn on the replica count, autoscaling, the pod disruption budget and topology spread. The multi-pod behaviour already exists in the code and runs harmlessly on a single pod today:

- **The scheduler lock** (see §5.3) — however many pods there are, only one runs a given sweep.
- **The database-backed session store** — sessions are shared across pods.
- **The single-active-session registry** — a user has one session regardless of which pod they land on.
- **Periodic settings cache refresh** — a change made on one pod reaches the others within seconds.

Measure two things before you make the move. The database connection pool is sized **per pod**, so pool size multiplied by replicas must stay within the database's connection limit; and if you use synthetic monitoring, every pod runs its own k6 subprocesses, so memory demand grows linearly with replicas.

---

# 18. Configuration Reference

Site Monitor has two kinds of setting, and it is important not to confuse them. **Static configuration** comes from files and environment variables and needs a restart to change. **Live settings** live in the database and are changed from the interface, taking effect immediately. This section gives the rule first, then both lists.

## 18.1 Precedence and Where a Setting Comes From

```
Highest precedence
      |
      +-- app_settings table         ->  Management -> Settings screen, IMMEDIATE
      |                                  (about 180 curated keys)
      +-- environment variable       ->  ConfigMap / Secret / .env, NEEDS A RESTART
      |
      +-- application.properties     ->  the code default
Lowest precedence
```

A row in the database is an **override**: delete it and the value falls back to the environment variable or the file default. So not seeing a key in the settings table does not mean there is no setting — it means it is running on its default.

If you run several pods, changes propagate quickly rather than instantly: settings caches refresh roughly every ten seconds, and the maintenance window cache every thirty.

The application logs an **effective configuration** block on every start-up. It is grouped by category, and each line ends with a label saying where the value came from — default, file, environment or database. Reading that block is the fastest way to solve "I changed this setting and nothing happened". Secrets are masked; encrypted values are never decrypted for the log, only reported as set or unset.

There are three configuration profiles: the default file (leaning towards development), the production profile (Kubernetes — database-backed sessions, secure cookies, file logging), and a local profile that serves a pre-built interface from a single port.

Real values are never committed; you work from the example files provided.

> **Environment variable names from before the product was renamed still work.** The application-specific prefix changed, but each key is read through a fallback chain: the current name first, then the previous name, then the default. When a variable with an old name is found, a warning is logged once at start-up. To finish the migration, update the names in the ConfigMap and the Secret; both names are valid in the meantime.

## 18.2 Static Configuration

| Parameter | Default | What it does |
|---|---|---|
| `site.monitor.warning-days` | 30 | The day the warning band starts |
| `site.monitor.parallel-workers` | 20 | Concurrent checks |
| `site.monitor.check-timeout-seconds` | 6 | TLS socket timeout |
| `site.monitor.scheduler.cron` | `0 0 * * * *` | The hourly sweep |
| `site.monitor.scheduler.stale-minutes` | 65 | When a domain counts as stale |
| `site.monitor.scheduler.lock-ttl-minutes` | 10 | Distributed lock lifetime |
| `site.monitor.scheduler.cleanup-cron` | `0 0 3 * * *` | Nightly clean-up and rollup |
| `site.monitor.alert.default-warning-days` | 30 | Warning threshold |

| Parameter | Default | What it does |
|---|---|---|
| `site.monitor.alert.default-high-days` | 15 | High threshold |
| `site.monitor.alert.default-critical-days` | 7 | Critical threshold |
| `site.monitor.alert.default-realert-hours` | 24 | Reminder interval |
| `site.monitor.cache.crl-ttl-hours` | 1 | CRL cache lifetime |
| `site.monitor.dns.query-timeout-ms` | 2000 | DNS query timeout |
| `site.monitor.dns.resolvers` | public resolvers | Used for propagation checks |

Capacity and resources:

| Parameter | Default | What it does |
|---|---|---|
| `SCHEDULING_POOL_SIZE` | 8 | The scheduled-job pool; the frequent sweeps and cache refreshes must all fit |
| `TOMCAT_MAX_THREADS` | 100 | The HTTP thread ceiling — deliberately below the framework default so it stays in proportion to the connection pool |
| `DB_POOL_MAX` / `DB_POOL_MIN` | 25 / 10 | Database connection pool, **per pod** |
| `EXECUTOR_CORE_SIZE` / `EXECUTOR_MAX_SIZE` | 20 / 50 | The check thread pool |
| `JAVA_OPTS` | see the chart | Sizes the heap against the container limit and exits on out-of-memory |

Security and networking:

| Parameter | What it does |
|---|---|
| `SITE_MONITOR_SECRET_KEY` | The encryption key for stored mail, directory and synthetic-monitoring secrets. **Mandatory in production**, must be stable and identical across pods; change it and stored secrets become unreadable |
| `CLIENT_IP_HEADERS` | The ordered list of headers to read the real client address from |
| `SPRING_SESSION_STORE_TYPE` | Database-backed in production, in-memory locally |
| `SITE_MONITOR_USERNAME` / `SITE_MONITOR_PASSWORD` | The bootstrap administrator account |
| `HTTP_PROXY_HOST` / `HTTP_PROXY_PORT` / `NO_PROXY` | Outbound proxy and its exclusions |
| `TLS_MODE` | `browser` by default, which presents a browser-like TLS fingerprint so that WAFs do not reset the connection |

Logging:

| Parameter | What it does |
|---|---|
| `logging.level.com.sitemonitor` | The application log level |
| `LOG_TIMEZONE` | The time zone used for log timestamps; business timestamps stay in UTC |

## 18.3 Identity and Lockout

| Parameter | Default | What it does |
|---|---|---|
| `site.monitor.lockout.failures-needed` | `5,3,2,1` | Failures required at each stage |
| `site.monitor.lockout.durations-seconds` | `30,120,600,1800` | The wait at each stage |
| `site.monitor.lockout.permanent-failures` | 5 | When the permanent lock applies |
| `site.monitor.remember-me.validity-seconds` | 604800 | "Remember me" lifetime, seven days |
| `PASSWORD_MIN_LENGTH` / `MAX_LENGTH` / `HISTORY_COUNT` | 6 / 64 / 5 | Password policy |

## 18.4 Email

| Parameter | What it does |
|---|---|
| `SITE_MONITOR_EMAIL_ENABLED` | Turns email on |
| `SPRING_MAIL_HOST` / `SPRING_MAIL_PORT` | Mail server and port |
| `SPRING_MAIL_USERNAME` / `SPRING_MAIL_PASSWORD` | Sending account credentials |
| `SITE_MONITOR_EMAIL_FROM` | The sender address |
| `APP_BASE_URL` | The base address used in email links |

These are enough to start. After installation, managing mail from **Settings → SMTP** is far more practical — changes there are written to the database, need no restart, and the screen can send a test message.

## 18.5 Live Settings

About 180 keys can be changed from **Management → Settings** while the application is running. The keys are not free-form; they come from a curated catalogue, so a setting you cannot see on screen cannot be changed live. The groups:

| Group | What it governs |
|---|---|
| `general` | Application base address, system administrator email, allowed origins, problem-reporting switches, environment name (version panel, deployment history and Grafana label; falls back to Helm's `APP_ENVIRONMENT`, then to an automatic value) |
| `security` | The corporate certificate authority bundle and automatic authority pinning |
| `branding` | Application name, tab title, sign-in text, primary colour, logo, announcement banner |
| `monitoring` | Per-type alert switches, internal and loopback target policy, DNS resolvers and timeouts, registration lookup addresses, page crawl limits |
| `scripted` | Synthetic monitoring pool, timeout ceilings, the k6 binary path, hard-coded-secret and syntax policies |
| `frequency` | Default interval, timeout and slow thresholds for new monitor forms |
| `outage` · `scheduler` | Bulk network outage thresholds, stale-check period |
| `storm` | Alert storm threshold unit, window and grouping behaviour |
| `login-anomaly` | Failed sign-in detection rules and recipients |
| `weekly` | Weekly report scoring weights |

| Group | What it governs |
|---|---|
| `logging` | The application log level — changeable live |
| `retention` | Retention period per table, batch size, rollup and the legal hold switch |

Three families live on their own screens: **SMTP**, **Directory** and **Retention**. Proxy settings are deliberately not live — they come only from environment variables, because changing network egress from a web interface would soften a corporate security boundary.

---

# 19. Release Information

| Item | Value |
|---|---|
| Current version | 20.128.0 |
| Document date | August 2026 |
| Java | 25 (LTS) |
| Spring Boot | 4.1.0 (Spring Framework 7, Jakarta EE 11, Hibernate 7, Tomcat 11) |
| BouncyCastle | 1.78.x |
| React / Vite | 18.3 / 5.4 |
| PostgreSQL | 16 or later |
| Container image | `ghcr.io/<owner>/site-monitor` |
| Helm chart | `ghcr.io/<owner>/sitemonitor-chart` |
| Languages | Turkish and English |
| Licence | Corporate use |

The version number comes from a single `VERSION` file and increments automatically from conventional commit prefixes:

| Prefix | Increment | Example |
|---|---|---|
| `feat:` | Minor | `1.4.2` → `1.5.0` |
| `fix:` / `chore:` / `refactor:` | Patch | `1.4.2` → `1.4.3` |
| Breaking change | Major | `1.4.2` → `2.0.0` |

The version fields inside the build files do not match this number and are not expected to — nothing reads them; the root `VERSION` file is the single source of truth.

## 19.1 Recent Highlights

The 20.x series is where the product matured under the Site Monitor name with its extended monitoring family. The highlights:

- Shareable deep links and consistent pagination across every list view: what you see on screen lives in the address bar, and no list ever draws more than two hundred records at once (see §14.1).

- Alert History rebuilt: server-side search, filters by level, team and ownership, a statistics strip, collapsible grouping by subject, "open for" and "recurred" badges, CSV export, and recognition of all twenty-eight alert types.
- A **mandatory reason note** when acknowledging or resolving an alert — the record now answers "why was this closed?" rather than just "who closed it?".
- The weekly availability email now carries a detailed outage report as a PDF attachment, with charts and status colouring.
- Retention managed from a single catalogue: a period per table, a dry run, a run history and a legal hold switch (see §15.8).
- Branding and email refresh: the brand mark across the interface, the tab icon and emails, with templates rebuilt to render correctly in corporate mail clients.
- The monitoring family completed: page integrity, synthetic k6 journeys and domain registration monitoring, with response charts up to 90 days.
- Maintenance windows with daylight-saving-safe recurrence, guaranteed silence and uptime exemption.
- Alert storm grouping and bulk network outage suppression to keep notification noise under control.
- System Health extended with database analytics, sign-in time series and an activity heat map.
- Hardening for OpenShift: probe groups, readiness gating at start-up and a liveness check independent of the database.

### 19.2 The Platform Upgrade

- Runtime: Java 21 to 25 (LTS); Spring Boot 3.3 to 4.1, bringing Spring Framework 7, Jakarta EE 11, Hibernate 7 and Tomcat 11.
- The JSON stack was upgraded with the API contract preserved by controller tests.
- The test framework moved to the current annotations, and the coverage and code-generation tooling was raised to versions compatible with the new JDK.
- Operational note: because the session schema changed, users may need to sign in once after the first deployment.

### 19.3 Lasting Gains from Earlier Generations

- Identity: directory sign-in, automatic provisioning on first sign-in, and multi-team permission scope.
- A single team model, with card-based team management.
- Weekly reports, approval by email link and a Friday reminder.
- Robustness: HTTP connection leaks closed, a global exception handler, error boundary layers and resilient parallel loading.
- Performance: escalation lookups batched at the start of a sweep, a single cache eviction at the end, and asynchronous mail retry.

---

## 20. Production Readiness Checklist

The default configuration is meant for local development and is deliberately insecure; the production profile and your environment variables must override it. Work through this list before you go live.

## 20.1 Mandatory Security Variables

| Variable | Why | Insecure default |
|---|---|---|
| `SITE_MONITOR_SECRET_KEY` | Stored directory and mail passwords are encrypted with it. Left empty, it falls back to a built-in development key. At least 32 random characters, and permanent. | empty → development key |
| `SITE_MONITOR_USERNAME` / `SITE_MONITOR_PASSWORD` | The bootstrap administrator credentials | `user` / `password` |
| `CORS_ALLOWED_ORIGINS` | Permit only your production hosts | local development hosts |
| `COOKIE_SECURE=true` | Session cookie over HTTPS only | `false`, though `true` in the production profile |
| `DB_PASSWORD` | The database password; mandatory in the production profile | — |

## 20.2 Recommended Settings

| Variable | Why |
|---|---|
| `APP_BASE_URL=https://<host>` | Email links use it; without it, links point at localhost |
| `SYSTEM_ADMIN_EMAIL` | The recipient for network outage and system notifications |
| `SITE_MONITOR_EMAIL_ENABLED=true` plus SMTP | With email off, alert notifications are silently not sent |
| `SPRING_SESSION_STORE_TYPE=jdbc` | Sessions survive pod restarts |
| Password policy variables | The defaults are production-appropriate; raise them to match your corporate policy |

## 20.3 Access and Permission Checks

- The bootstrap administrator gate: the account named in the configuration always reaches the mail, directory, secret, database and general settings. Renaming the account does not lock you out, and creating another user called "admin" does not get anyone in.
- The permission matrix: the administrator role is locked and cannot be revoked. Destructive operations are marked sensitive and ask for confirmation when granted; two system-wide operations are additionally administrator-protected.
- Default permissions are seeded on first start-up, and new permissions introduced by later releases are backfilled into existing databases with administrator customisations preserved.

## 20.4 Kubernetes and OpenShift Notes

- The health probes use separate groups: liveness is independent of the database, so a database wobble does not restart the pod, while readiness and startup refuse traffic until boot has finished.
- Deploy with `helm upgrade` and verify with a rollout status check. A generous CPU limit speeds up the cold start.
- If a corporate SSL-inspecting proxy breaks domain registration lookups, capture the proxy certificate chain from **Settings → Domain Diagnostics** and add it to the trust bundle. Some country-level domains do not offer a modern registration protocol at all and need a firewall opening for the legacy port.

## 21. Glossary

Terms are used consistently throughout this guide; the Turkish equivalents used in the interface are given where they differ.

| Term | Definition |
|---|---|
| Alert | The recorded event for a detected problem. In this guide "warning" refers only to the WARNING severity level. |
| Alert level | WARNING, HIGH or CRITICAL — how urgent the problem is. |
| Alert type | The class of problem: expiry, revoked, broken chain, mismatch, accessibility, port down, DNS failure, DNS changed, registration expiry and so on. |
| Acknowledgement | Marking an alert as seen. It stops the daily reminders; it does not close the alert. |
| Resolution | The problem is fixed, the alert closes and a resolution notice is sent. |
| Re-alert | The daily reminder sent for an open, unacknowledged alert. |
| Escalation | Widening the recipient list as the alert level rises. |
| Escalation contact | A person who receives a team's alerts, carrying a minimum level and an optional webhook. |
| Confirmation count | How many consecutive failed checks are required before an alert is raised. |
| Recovery count | How many consecutive successful checks are required before a monitor is declared healthy. |
| Alert storm | Many simultaneous outages collapsed into a single bulk notification. |
| Maintenance window | A planned downtime period during which alerts and notifications are fully suppressed for the target monitors. |
| Sweep | One scheduler pass across all targets. |
| Inventory | The register of monitored certificate domains. |
| Tier | The criticality band of a domain, from 1 (customer-facing production) to 4 (development and sandbox). |
| Monitor | An individual monitoring target outside the certificate sweep — HTTP, port, DNS, keyword, ping, page, synthetic or domain. |
| Uptime percentage | The proportion of successful checks for a target; checks made during a maintenance window are excluded. |
| Registration lookup | The protocols used to query domain registration data; the modern one is preferred and the legacy one is the fallback. |
| OCSP / CRL | Certificate revocation checking methods; OCSP is tried first, CRL is the fallback. |
| Distributed lock | The database lock that keeps a sweep running on exactly one pod. |
| Bootstrap administrator | The local administrator account defined at installation that always reaches the settings screen. |
| Permanent deletion | Removing a record and its data irreversibly; every delete in Site Monitor is permanent (switch "Active" off to pause instead). |
| Provisioning | Creating a directory user automatically on first sign-in and assigning their role and team. |
| Webhook | An HTTP notification to a Teams or Slack channel. |
| Inline image | An image embedded in an email by content reference, so it renders even when remote content is blocked. |

| Term | Definition |
|---|---|
| Rollup | Summarising raw check rows into daily and hourly tables before deletion, so the long-term trend outlives the raw data. |
| Retention | How many days of data a table keeps; the nightly job removes anything older. |
| Legal hold | A single switch that stops all automatic deletion, used during audit periods. |
| Bulk network outage | Deciding, when the error rate crosses a threshold, that the fault is our own egress rather than the targets, and suppressing individual alerts. |
| Certificate authority pinning | Trusting and storing the authority first seen on a server, then validating later checks against it. |
| Corporate authority bundle | Adding an SSL-inspecting proxy's certificate authority alongside the system roots as an extra trust anchor. |
| Registrable domain | The root form of a domain at which registration is held; registration lookups always happen at this level. |
| EPP status code | Status labels on a registration record, some of which indicate the domain is close to being lost. |
| Proxy exclusion suffix matching | An entry in the proxy exclusion list also covering its subdomains. |
| Request forgery protection | Validating every resolved address of a monitoring target against internal, loopback and cloud metadata ranges. |
| Deep link | A shareable address carrying the filter, page and open-window state of a screen. |
| Scoped administrator | An administrator limited to particular teams, who — unlike a global administrator — cannot reach the permission matrix, audit log or SQL console. |

---

*This document was prepared for Site Monitor v20.128.0, as of August 2026.*